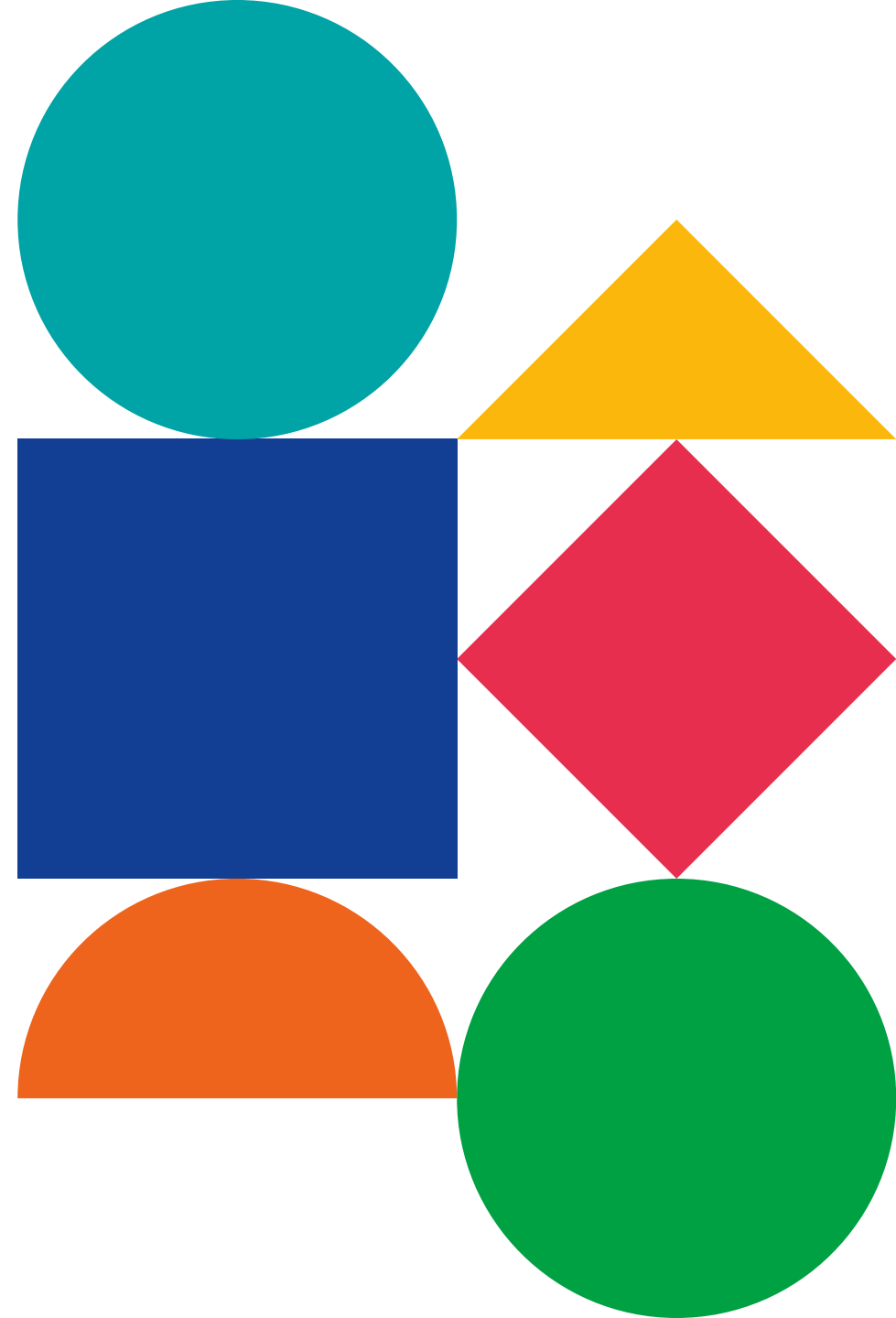


Versioning with Git & GitHub

For code and documentation - Day 2

Bruna Piereck

Dec 2025



REVIEW QUIZ

 **Kahoot!**



Uses case

- Covered in the course
 - Single user
 - Collaborating
- Not to be covered
 - Contribute to open-source project



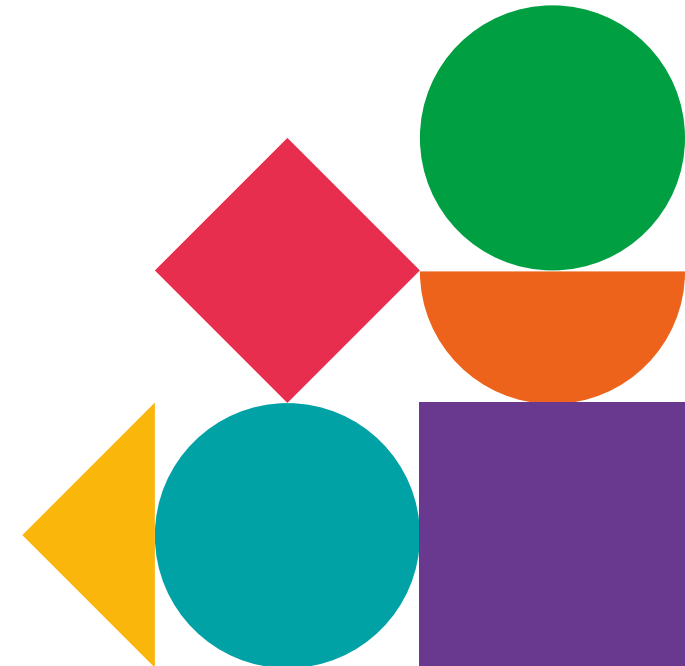
Back up

History

Recover previous state

Working remotely

Publication





Experimenting Risk Free

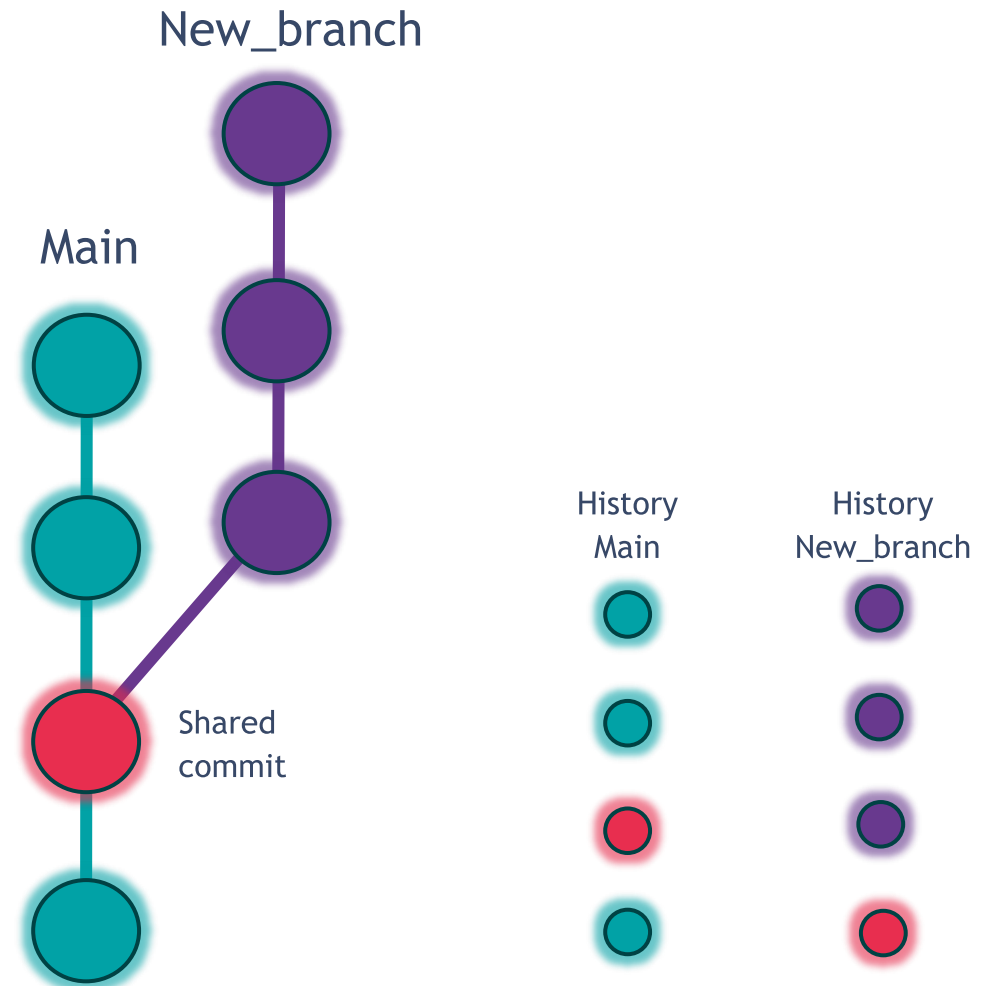
Alternative
timeline

Alternative history

```
user$ git branch <name>
```

Branch

- Alternative history of commits
- Has a specific name
- Independent timeline
- Shared commit between timelines



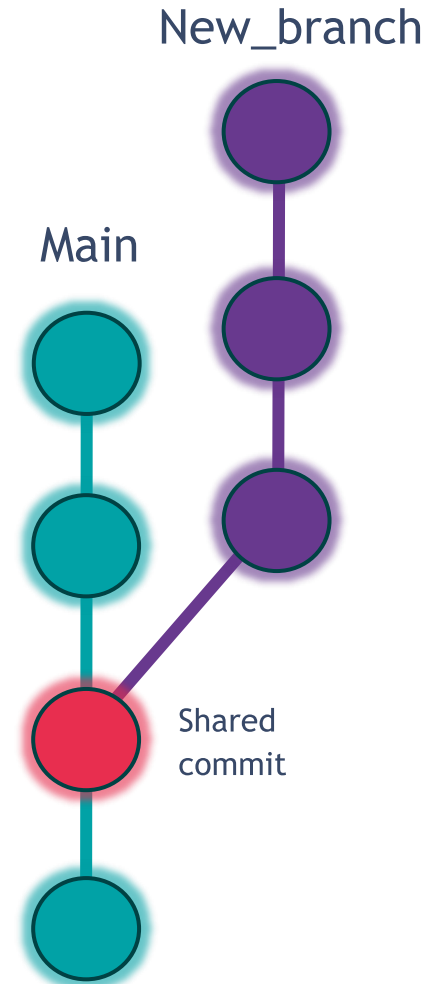
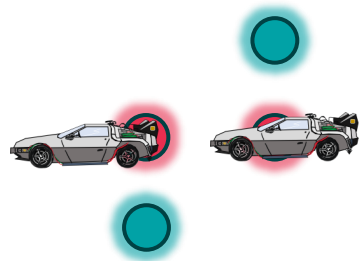
Alternative history

```
user$ git branch <name>
```

Utility

1. Experiment risk free
2. Collaborating in a project
3. Work in different tasks
4. Testing different possible solutions

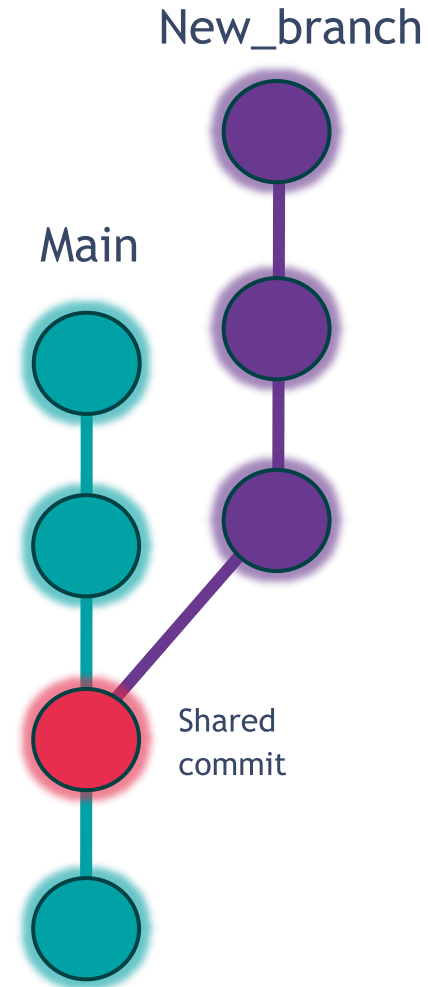
P.s.: Creating $\neq$ Working on



Moving between timelines

```
user$ git checkout <branch_name>
```

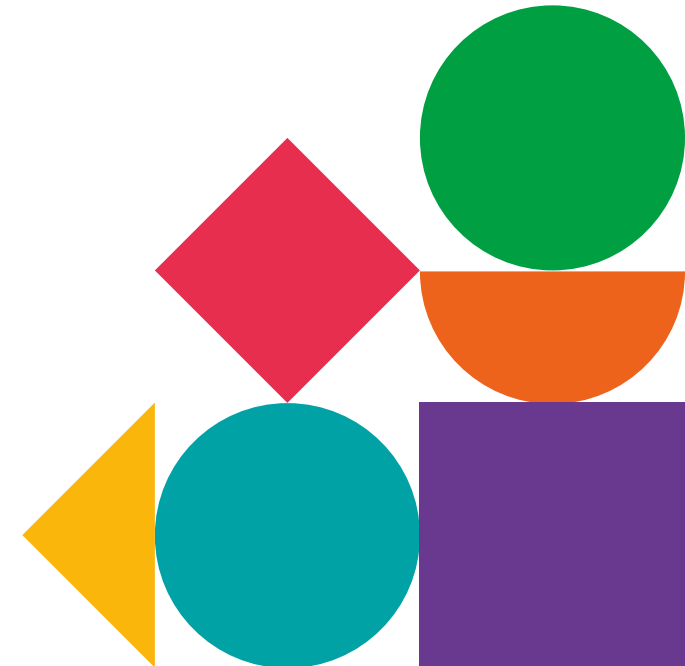
- Move between branches
- Move between commits
(!! Dangerous)



ACTIVITY

4. Collaborating & making use of branches

- Make sure you have a “working tree clean”
- Close all the files
- Go to your partner’s project
 - Sync all the content
 - Create your branch
`git branch <name>`
 - Checkout to your new branch
`git checkout <name>`
 - Write the concepts of Branching & Checkout
 - Write why branching can be useful
 - Commit and Push



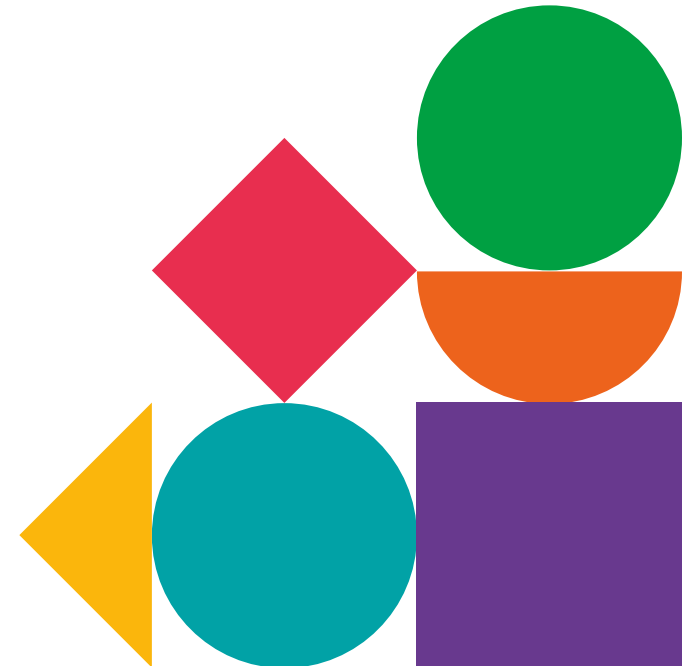
New branches & collaborations

```
user$ git branch -- list
```

- Check local branches

```
user$ git branch -a
```

- Check all branches
 - Local
 - Remote



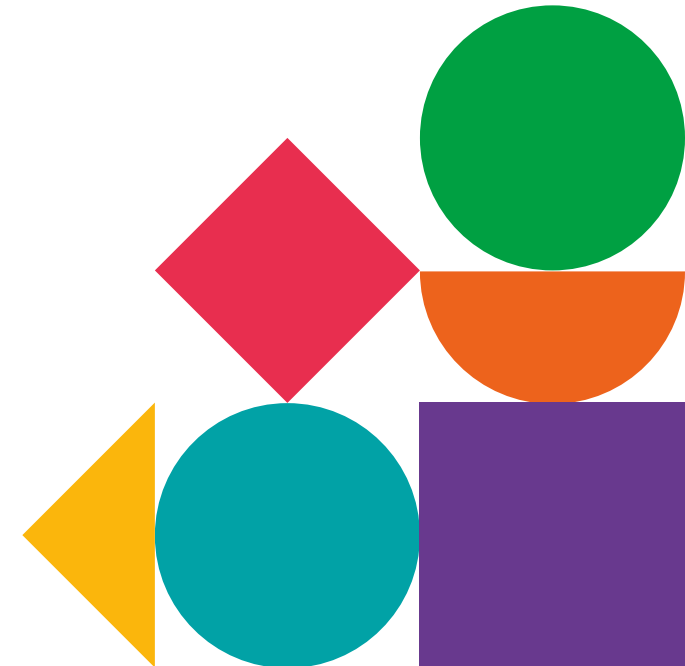
ACTIVITY

2. Go back to your project

- Sync the content
- Check the updates in the new branch your collaborator has created

```
user$ git branch - a
```

```
user$ git checkout <name>
```

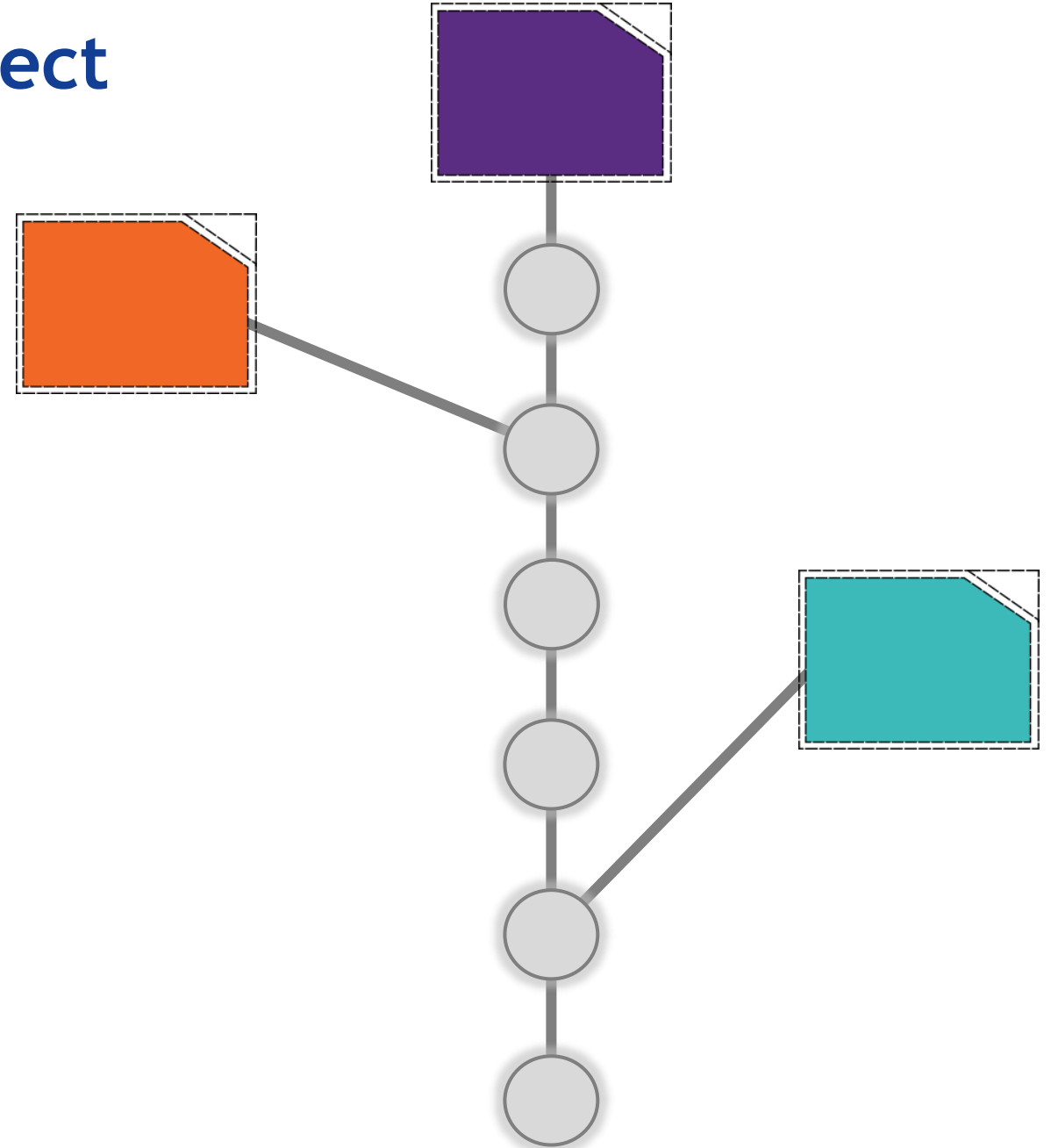
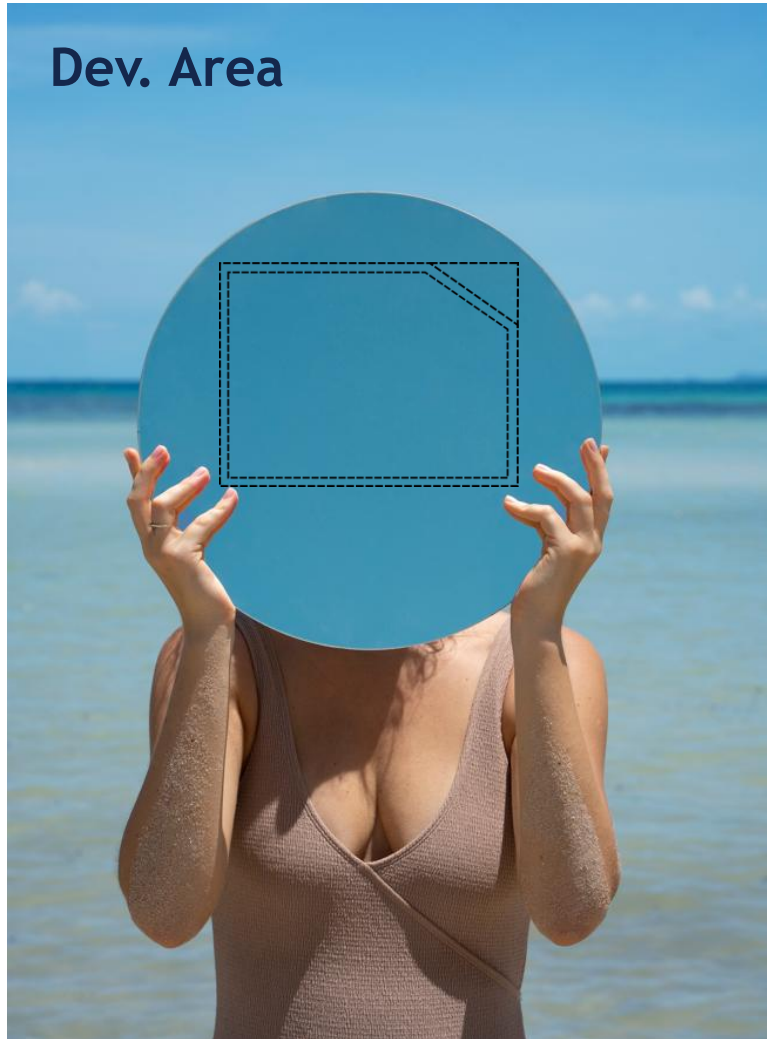


The “Mirror” effect

Dynamic
Dev. area

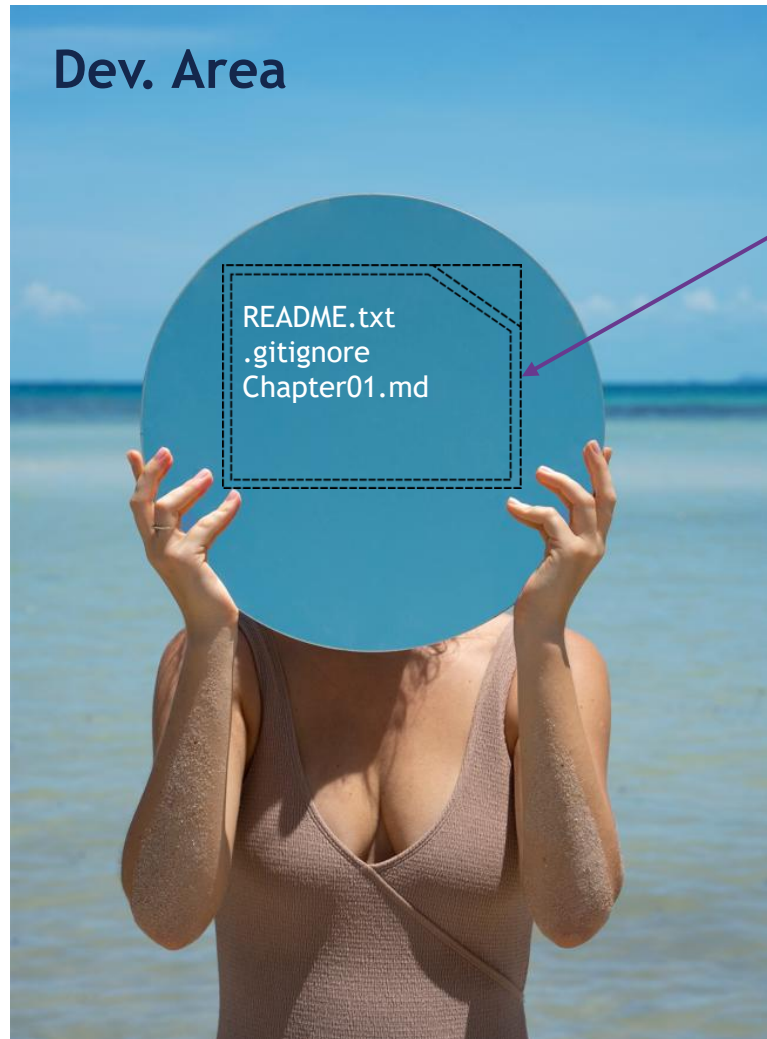


Branches: The “mirror” effect

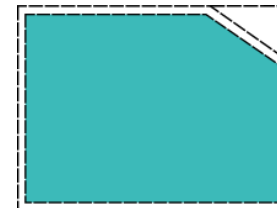


Branches: The “mirror” effect

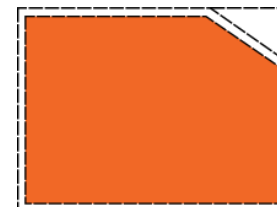
Git checkout vib_green



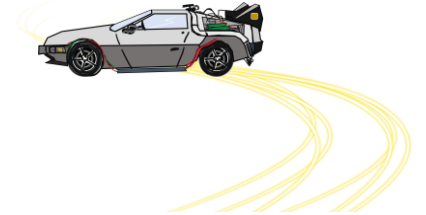
README.txt
.gitignore
Chapter01.md



README.txt
.gitignore
Chapter02.md

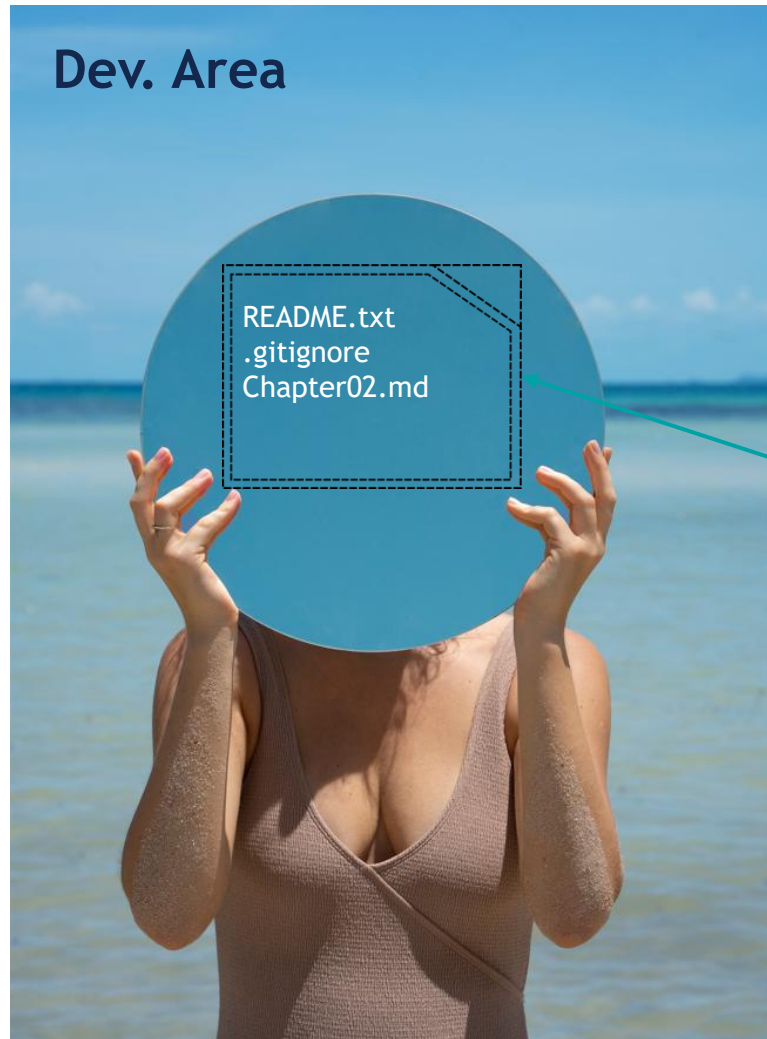


README.txt
.gitignore
Chapter01.md
Illustrations

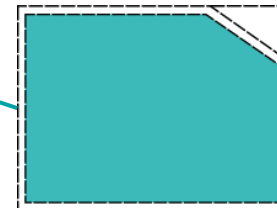


Branches: The “mirror” effect

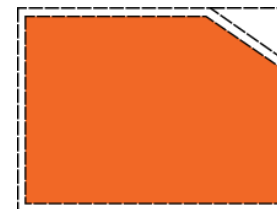
Git checkout vib_green



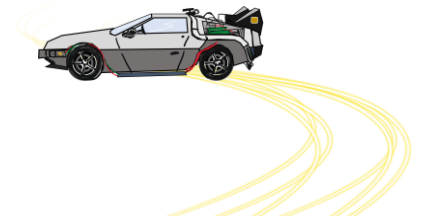
README.txt
.gitignore
Chapter01.md



README.txt
.gitignore
Chapter02.md



README.txt
.gitignore
Chapter01.md
Illustrations

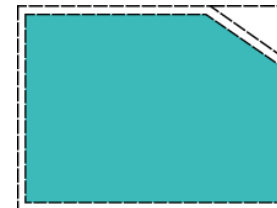


Branches: The “mirror” effect

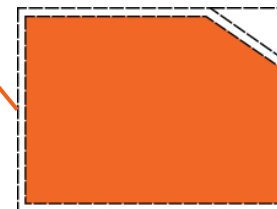
Git checkout vib_green



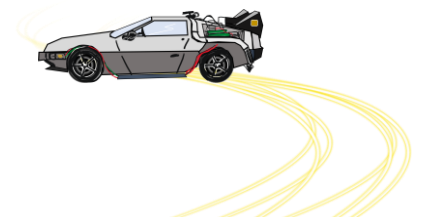
README.txt
.gitignore
Chapter01.md



README.txt
.gitignore
Chapter02.md



README.txt
.gitignore
Chapter01.md
Illustrations



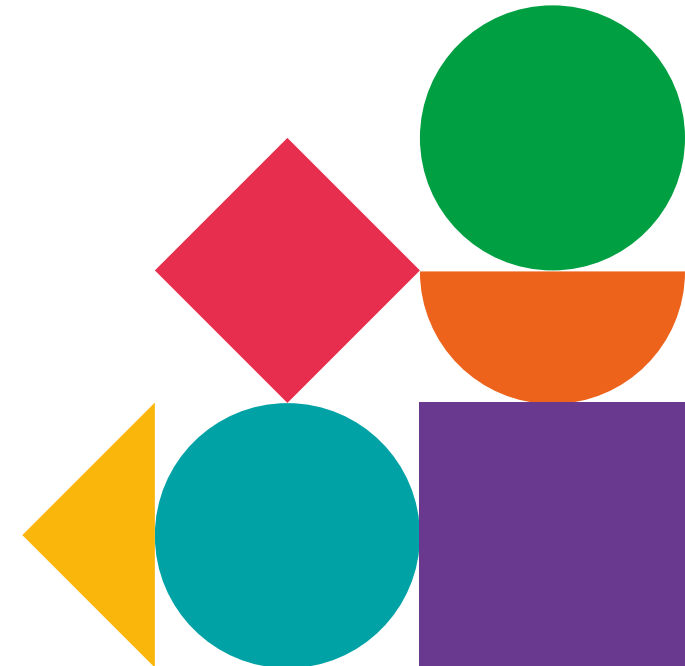


Coffee time

ACTIVITY

3. Compare content in both timelines

- Check your branch history
- Close all files
- Checkout to the main branch
`git checkout main`
- Check the history
 - Is there any differences?
- Open the files
 - Is there any information missing?
 - Why?



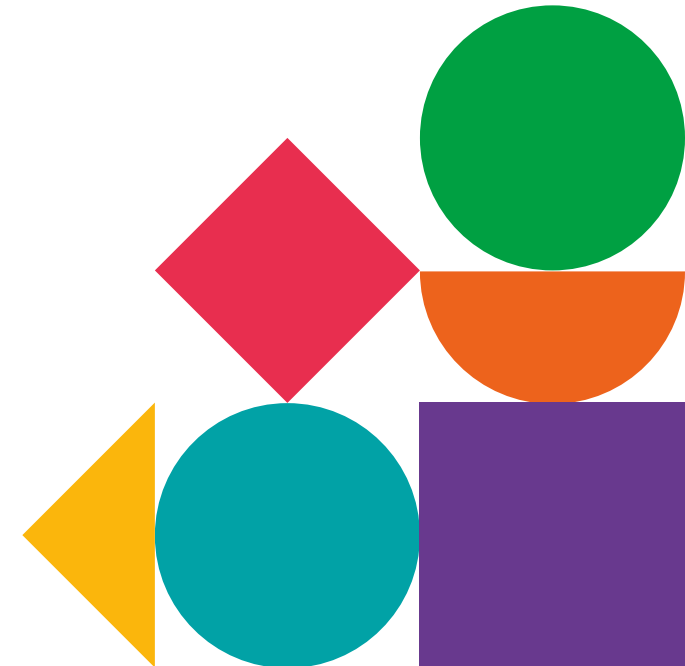
ACTIVITY

4. Working with branches

- Create a new_branch
- Write about the “mirror” effect
- Commit & Push

5. Comparing

- Close all files
- Move back to main
 - Is the content there?
 - Why?
- Can you see all branches in GitHub?



QUIZ

■ Why I can't see the changes in the file present in the Main Branch?

A. I lost what I just did?

B. It can only be found in the branch where I edited it

C. Git reloads my files based in the branch I am in

 B and C are correct

E. I have no idea



QUIZ

- How many copies of each file I have in the Dev. Area?

 Only 1.

B. I have 2, one for each branch

C. As many as branches I have

D. As many as versions I have

- Where are the other versions?

A. In another folder

B. I GitHub only

 In the .git folder



QUIZ

■ Where can I create a branch?

A. Only projects I own

☒ B. Projects I own or collaborate

C. Any repository

D. B and C are correct



QUIZ

- Can you see all branches your collaborators have created?

 Only the ones pushed to GitHub

B. Yes, I can see all

C. No, I can't see them unless invited for collaboration

D. I can see them on GitHub always



Branches review

```
user$ git branch -- list
```

```
-a
```

```
<name>
```

List the local branches

List all branches, local & remote

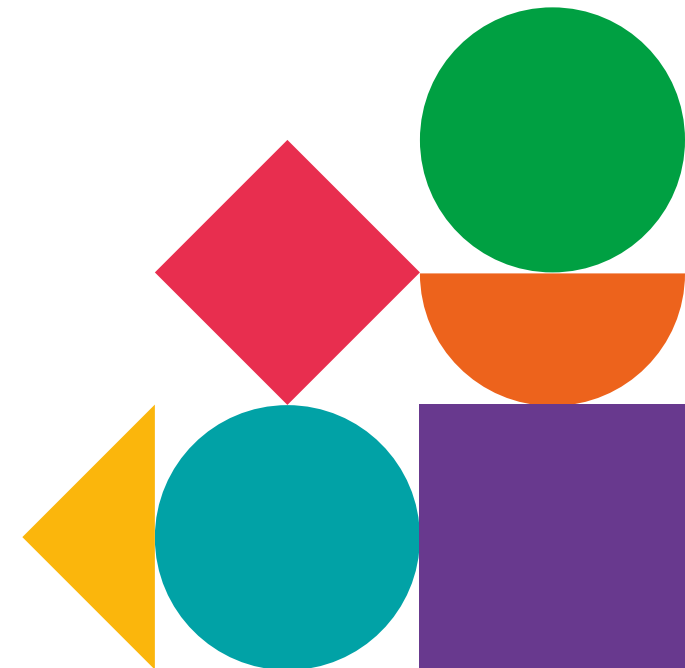
Create a new branch

```
user$ git checkout <name>
```

```
-b <name>
```

Go to branch

Create and go to new branch





Change the past at your own risk

Checkout to an
old commit



Detached head

```
user$ git checkout <commit_ID>
```

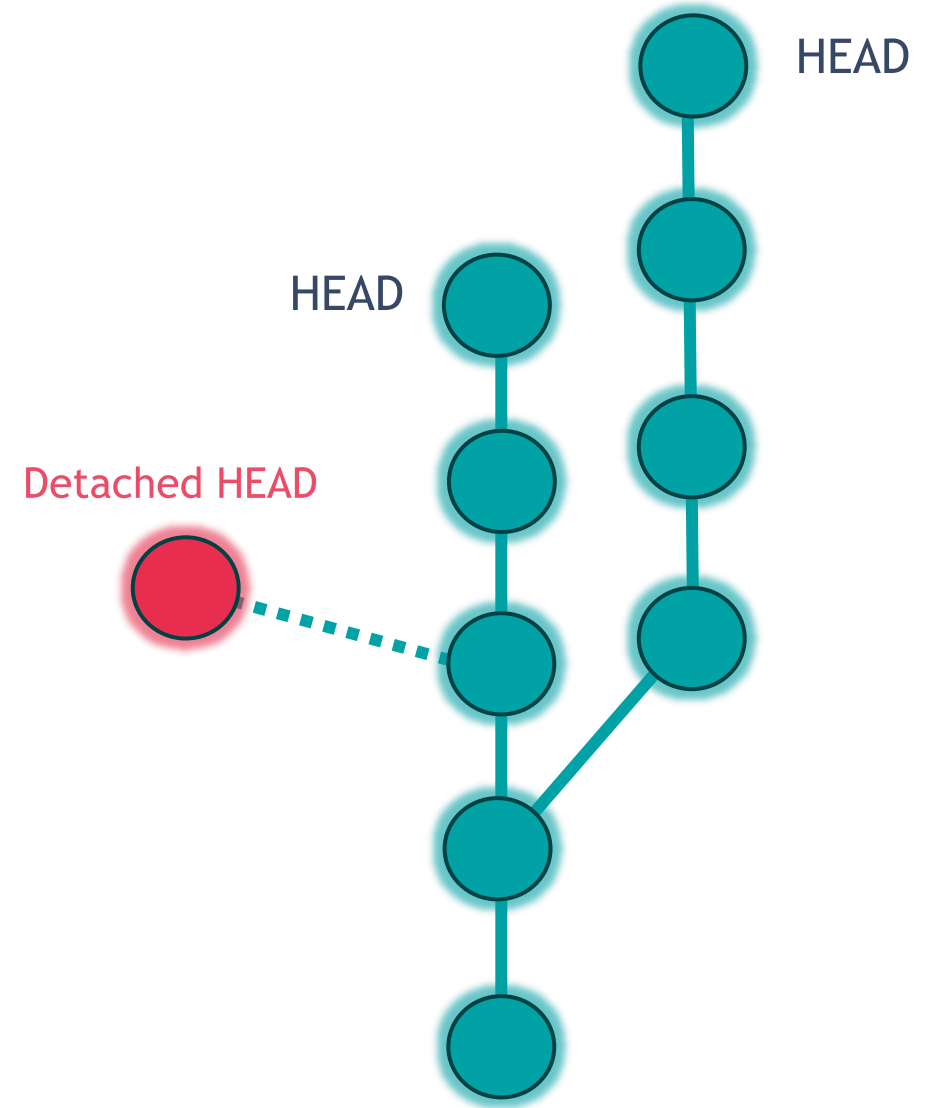
- You can't change the past
- You CAN create “an alternative future”
 - Be careful !!

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using `-c` with the switch command. Example:

```
git switch -c <new-branch-name>
```

```
user$ git switch -c <name_new_branch>
```



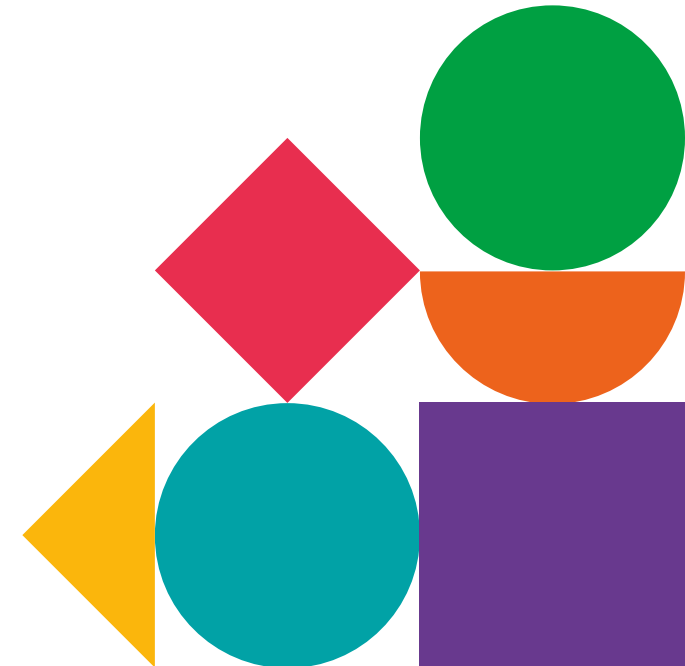
ACTIVITY

6. Changing the past

- Move to an old commit
- Add some text (2 steps)
 - Commit without switching
 - *Check in the list of branches*
 - Commit and do switch - c
 - *Check the list of branches*
- What happens?

```
user$ git checkout <commit_ID>
```

```
user$ git switch -c <name_new_branch>
```



Branches review

```
user$ git branch -- list
```

```
-a
```

```
<name>
```

List the local branches

List all branches, local & remote

Create a new branch

```
user$ git checkout <name / ID >
```

```
-b <name>
```

Go to branch

Create and go to new branch

```
user$ git switch -c <name_new_branch>
```

Transform a detached HEAD in a branch

Merging

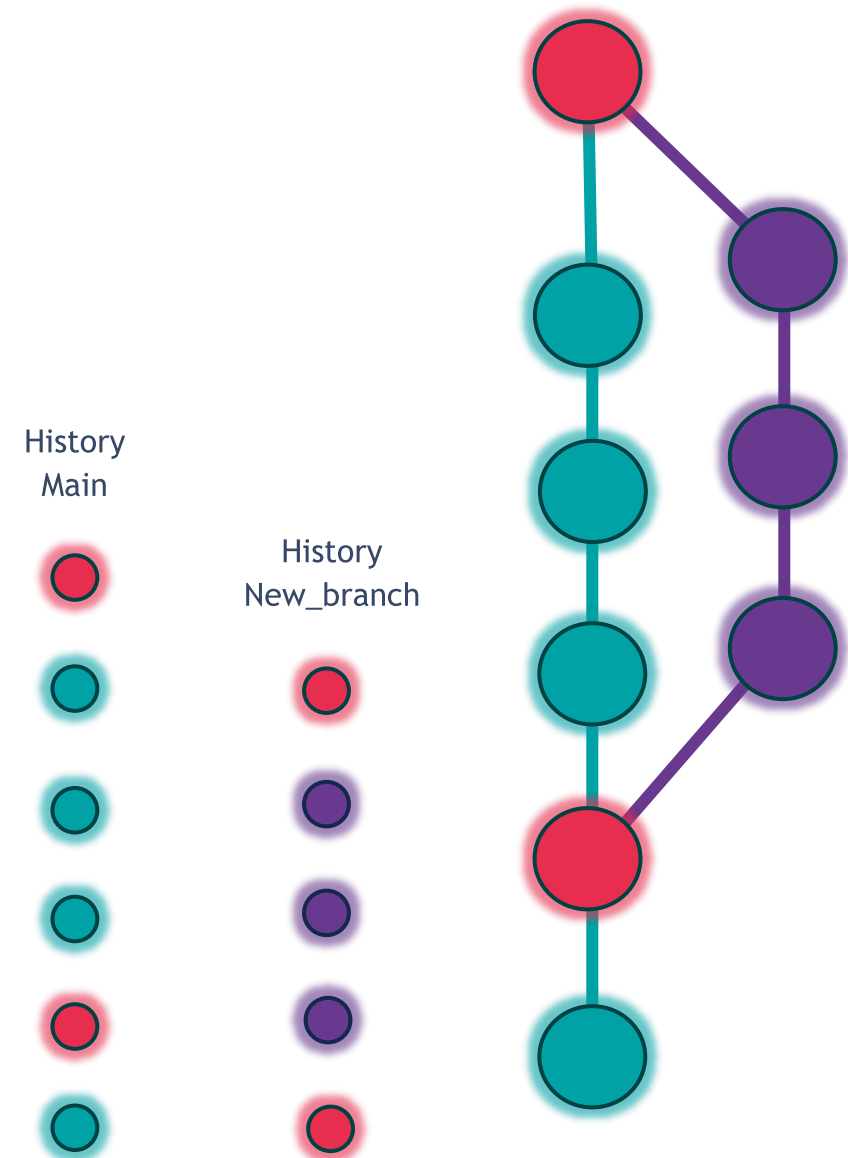
Branches come
together

Merging Branches

```
user$ git merge <branch_name>
```

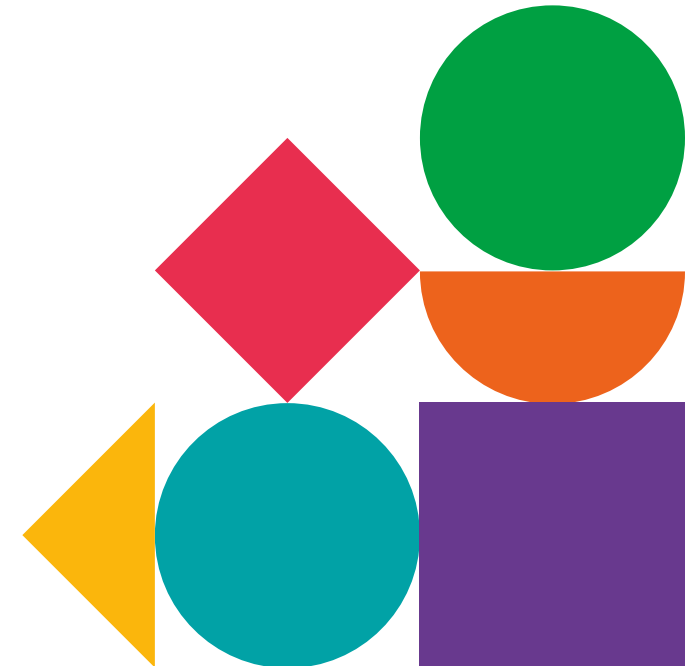
- Use cases
 1. Experiment worked out
 2. Finished collaboration
 3. Problem solved
- Too many branches
 - Clean up

```
user$ git branch -d <branch_name>
```



ACTIVITY

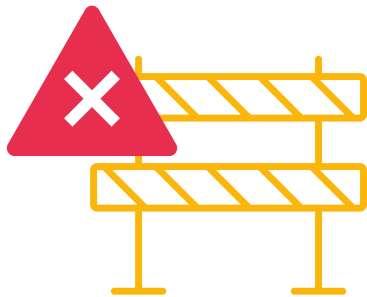
7. Create a new branch
 - Write about merging



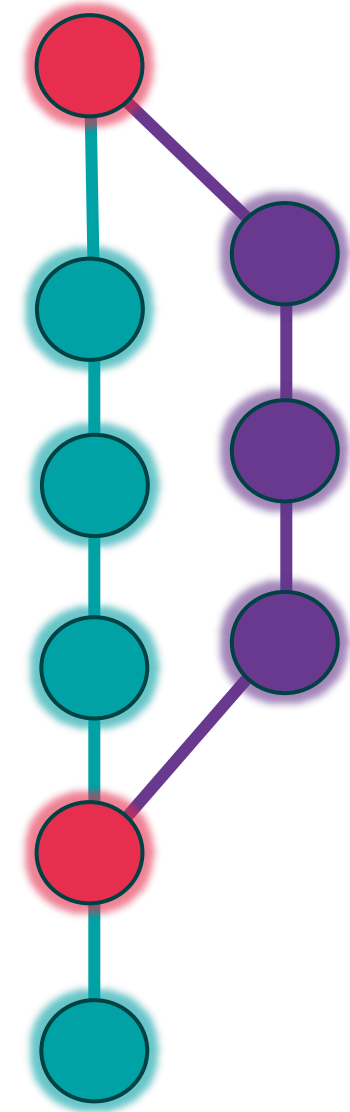
Merging Branches

```
user$ git merge <branch_name>
```

- Use cases
 1. Experiment worked out
 2. Finished collaboration
 3. Problem solved



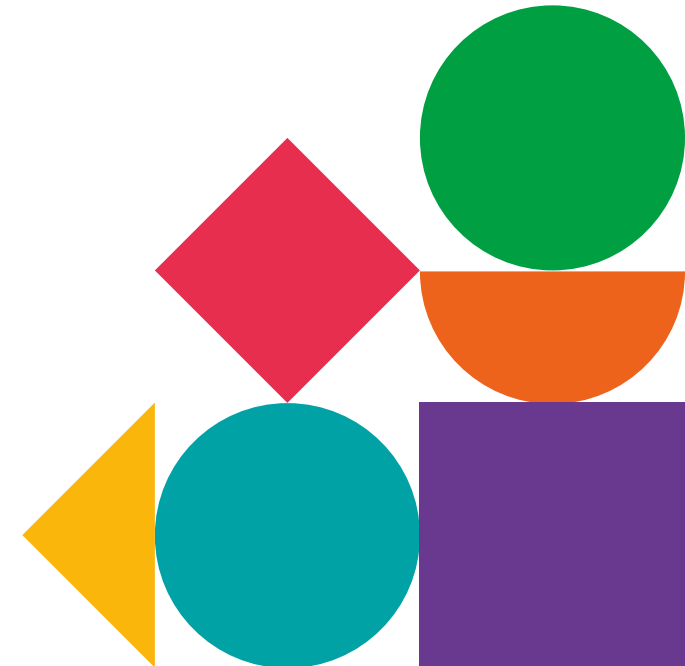
Challenge !!!
Solving conflicts



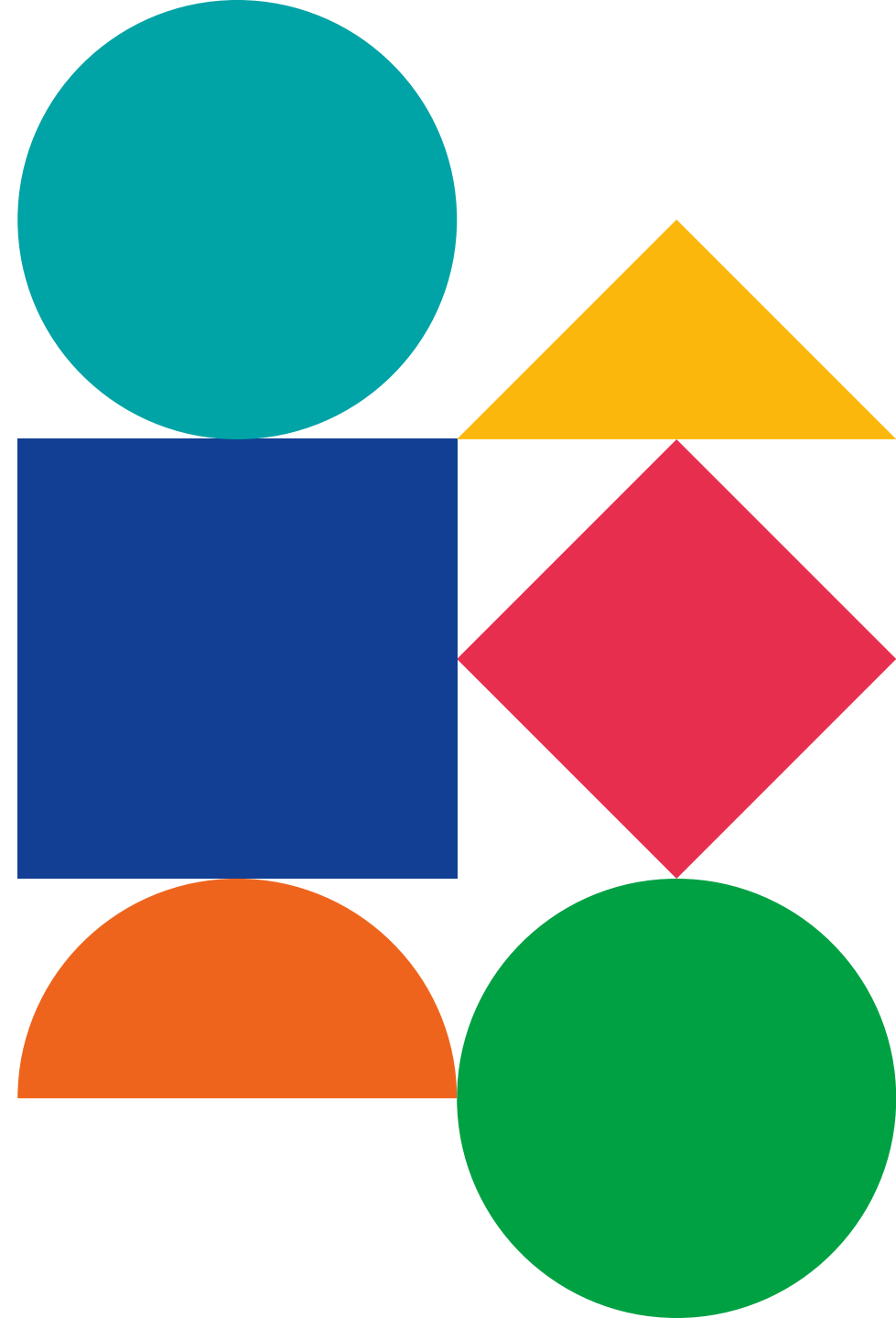
ACTIVITY

8. Merge one-by-one

- Merge all your branches with main
- Use conflict solving to keep only relevant information
- Think carefully of your commit messages
- Push your changes to GitHub



REVIEW



Review

```
user$ git branch <branch_name>
```

- Creates an alternative timeline

```
user$ git checkout <branch_name>
```

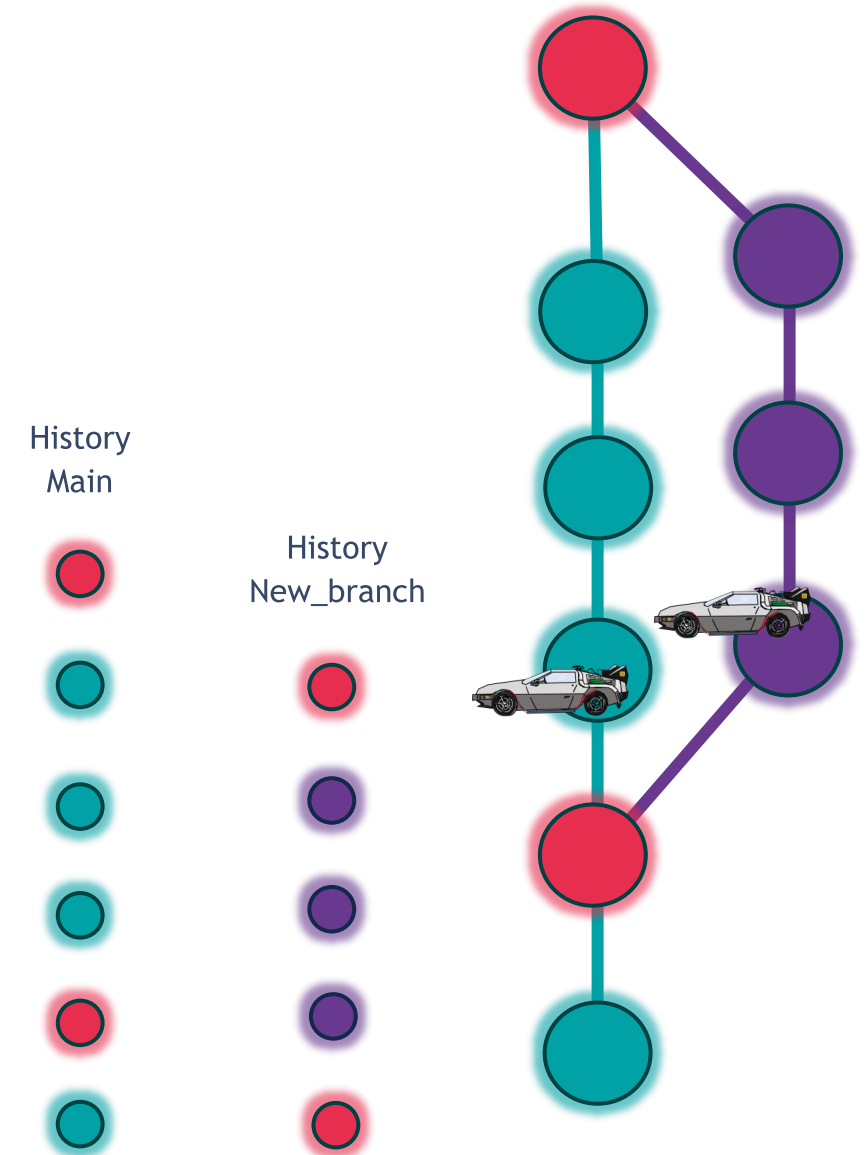
- Moves in the alternative timeline or old commit

```
user$ git checkout -b <branch_name>
```

- Creates and moves in the alternative timeline

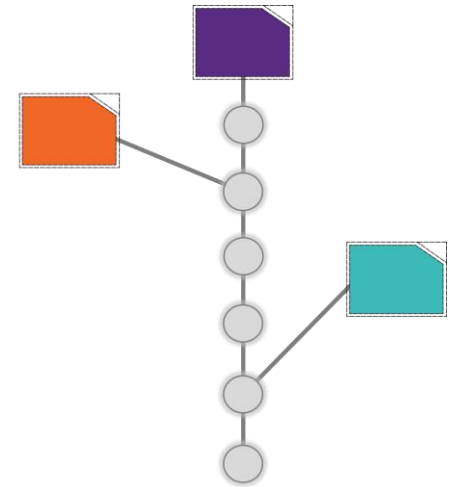
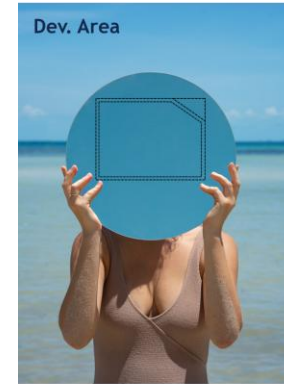
```
user$ git merge <branch_name>
```

- Merges two timelines in a shared commit point

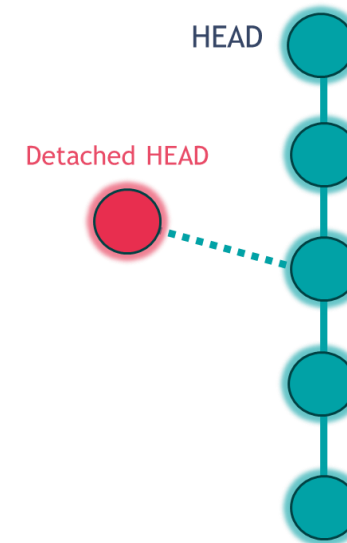


Review

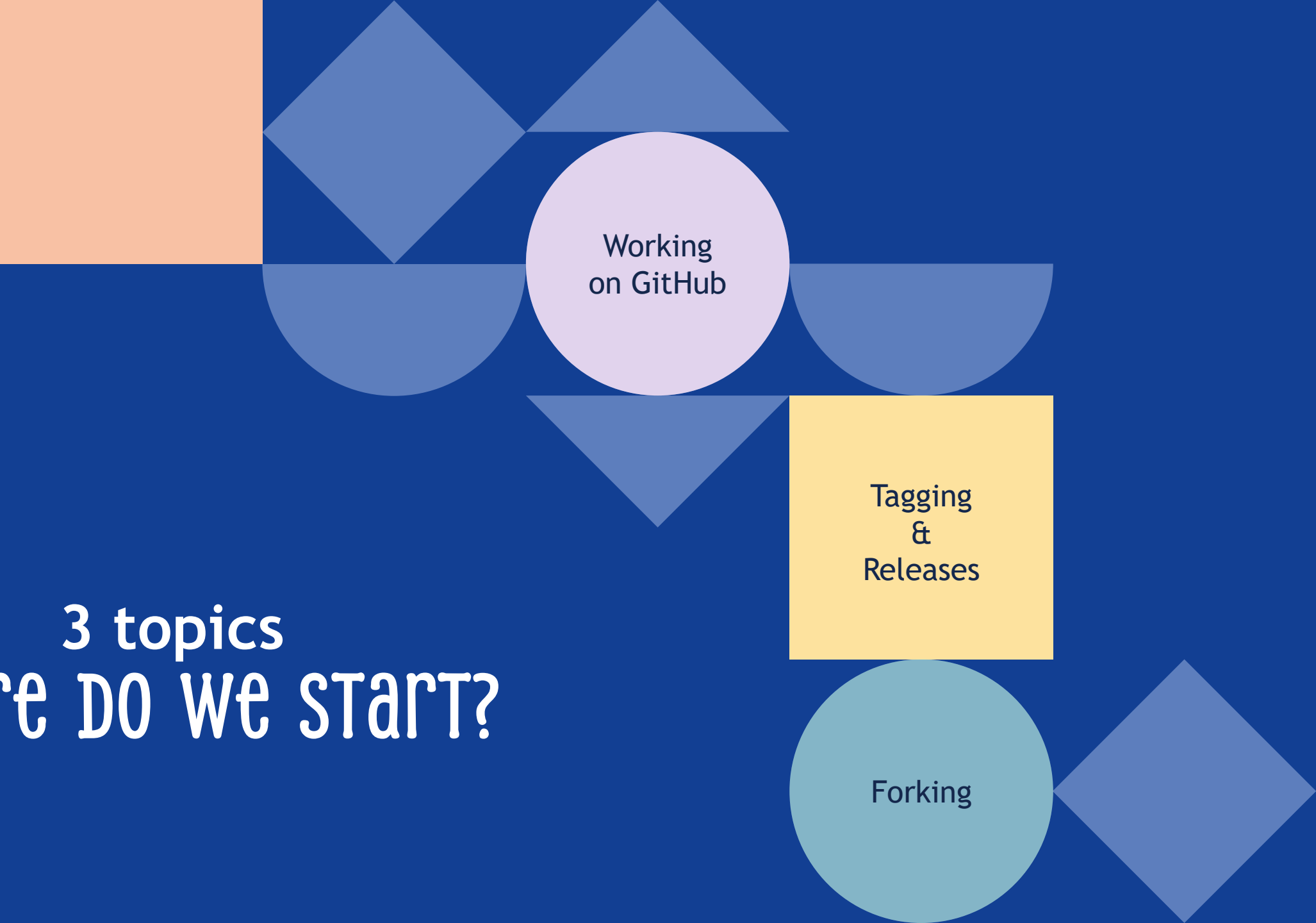
- The “mirror” effect
 - Folder Content == Branch you are working on



- Detached HEAD
 - Checking to old commit
 - A “floating” branch
 - Need to be connected



user\$ git switch -c <branch_name>

A diagram on a dark blue background. It features a central light purple circle containing the text "Working on GitHub". To its right is a yellow square containing the text "Tagging & Releases". Below the yellow square is a light blue circle containing the text "Forking". Various geometric shapes, including diamonds and semi-circles in shades of blue, are arranged around these central elements. A large orange square is in the top left corner.

Working
on GitHub

Tagging
&
Releases

Forking

3 topics
WHERE DO WE START?

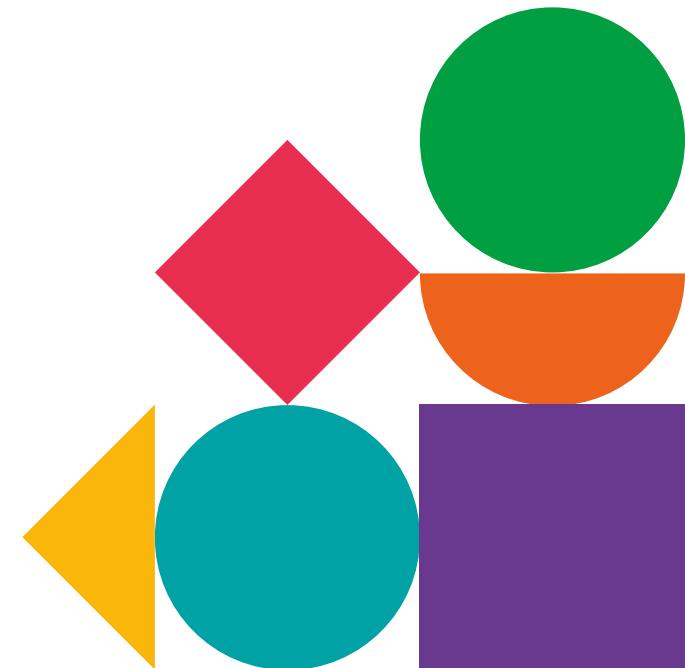


GitHub

Working
Online

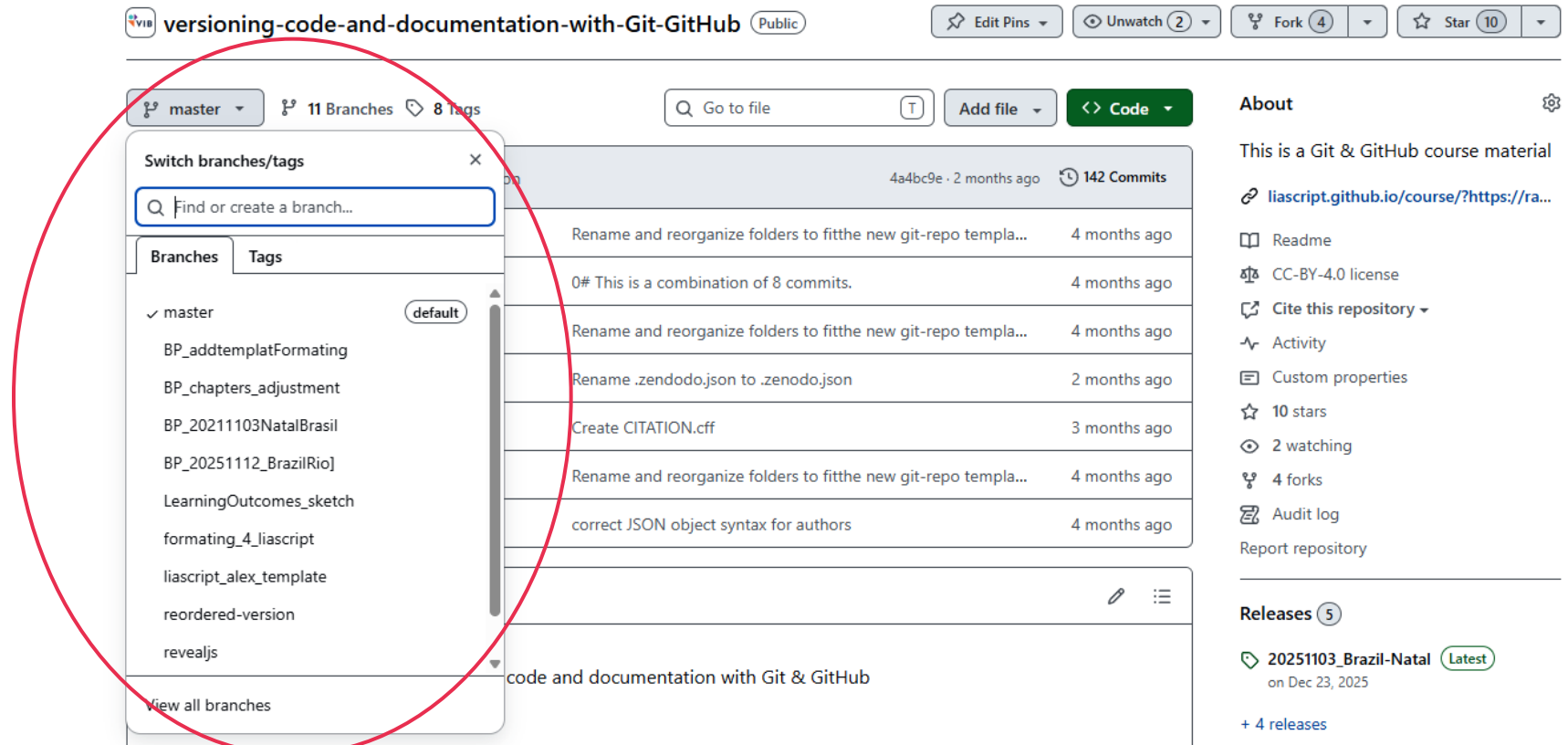
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



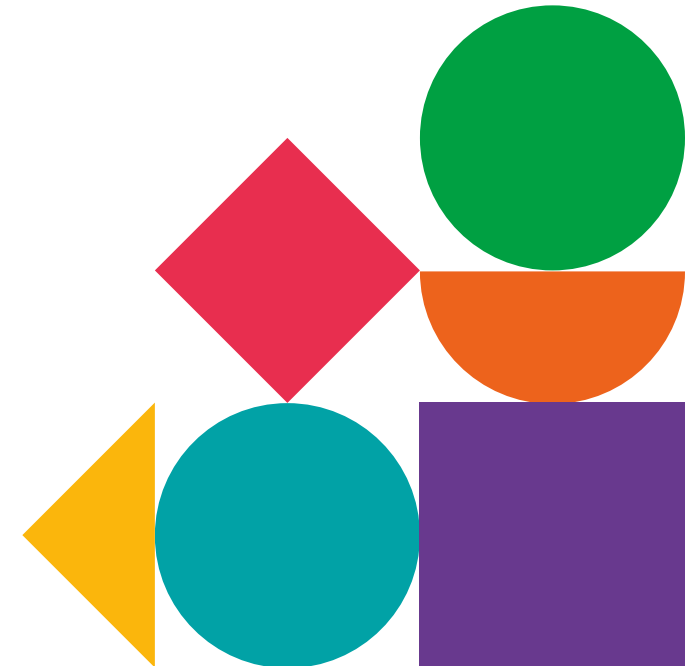
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits

 abotzki	Rename .zendodo.json to .zenodo.json	4a4bc9e · 2 months ago	🕒 142 Commits
data	Rename and reorganize folders to fit the new git-repo template...	4 months ago	
docs	0# This is a combination of 8 commits.	4 months ago	
exercises	Rename and reorganize folders to fit the new git-repo template...	4 months ago	
.zenodo.json	Rename .zendodo.json to .zenodo.json	2 months ago	
CITATION.cff	Create CITATION.cff	3 months ago	
LICENSE	Rename and reorganize folders to fit the new git-repo template...	4 months ago	

1 file changed +24 -23 lines changed

Search within code

+24 -23

@@ -151,7 +151,7 @@ Authors

```
151 - @orcid(Bruna Piareck)](http://orcid.org/00000-0001-5958-0669)
152 - @orcid(James Collier)](http://orcid.org/0000-0002-0020-421X)
153 - @orcid(Tuur Muyldermans)](http://orcid.org/0000-0002-3926-7293)
154 - - @orcid(Alexander Botzki)](http://orcid.org/0000-0001-6691-4233)
```

```
155
156 ## Citing this lesson
157
```

@@ -171,121 +171,121 @@ There are preparation steps for all, Windows, Linux and Mac computers.

```
171
172 Most things you need to get installed before hands, one or the other are optional tools to help with
visualization and for somethings you can ask the trainer by mail or arriving 30 min before the class
starts.
173
```

```
174 - [click here ](https://liascript.github.io/course/?https://raw.githubusercontent.com/vib-tcp/introduction-github/refs/heads/0P\_addtemplatFormatting/docs/tutorials/01\_Get\_ready\_for\_the\_course.md#1)
175
176 —
```

```
151 - @orcid(Bruna Piareck)](http://orcid.org/00000-0001-5958-0669)
152 - @orcid(James Collier)](http://orcid.org/0000-0002-0020-421X)
153 - @orcid(Tuur Muyldermans)](http://orcid.org/0000-0002-3926-7293)
154 + - @orcid(Alexander Botzki)](http://orcid.org/0000-0001-6691-4233)
```

```
155
156 ## Citing this lesson
157
```

```
171
172 Most things you need to get installed before hands, one or the other are optional tools to help with
visualization and for somethings you can ask the trainer by mail or arriving 30 min before the class
starts.
173
```

```
174 + [click here ](https://liascript.github.io/course/?https://raw.githubusercontent.com/vib-tcp/introduction-github/refs/heads/master/docs/tutorials/01\_Get\_ready\_for\_the\_course.md#1)
175
176 —
```

All time

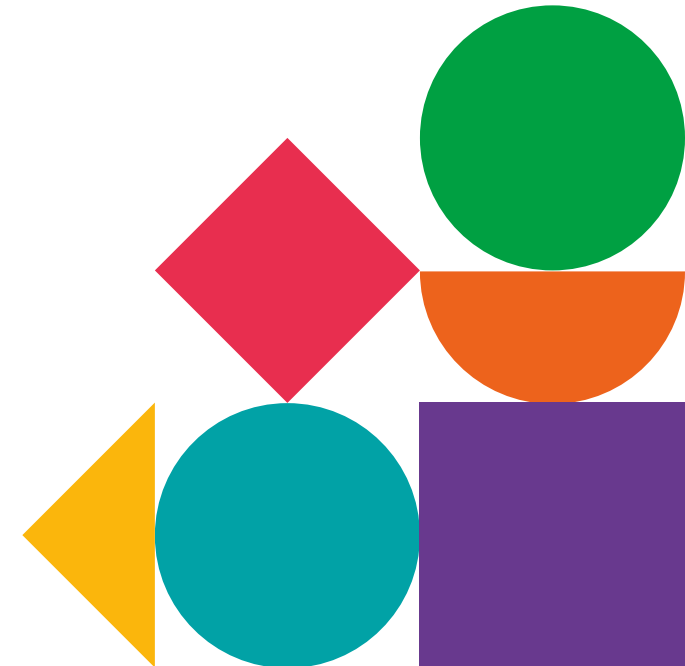
<>

<>

<>

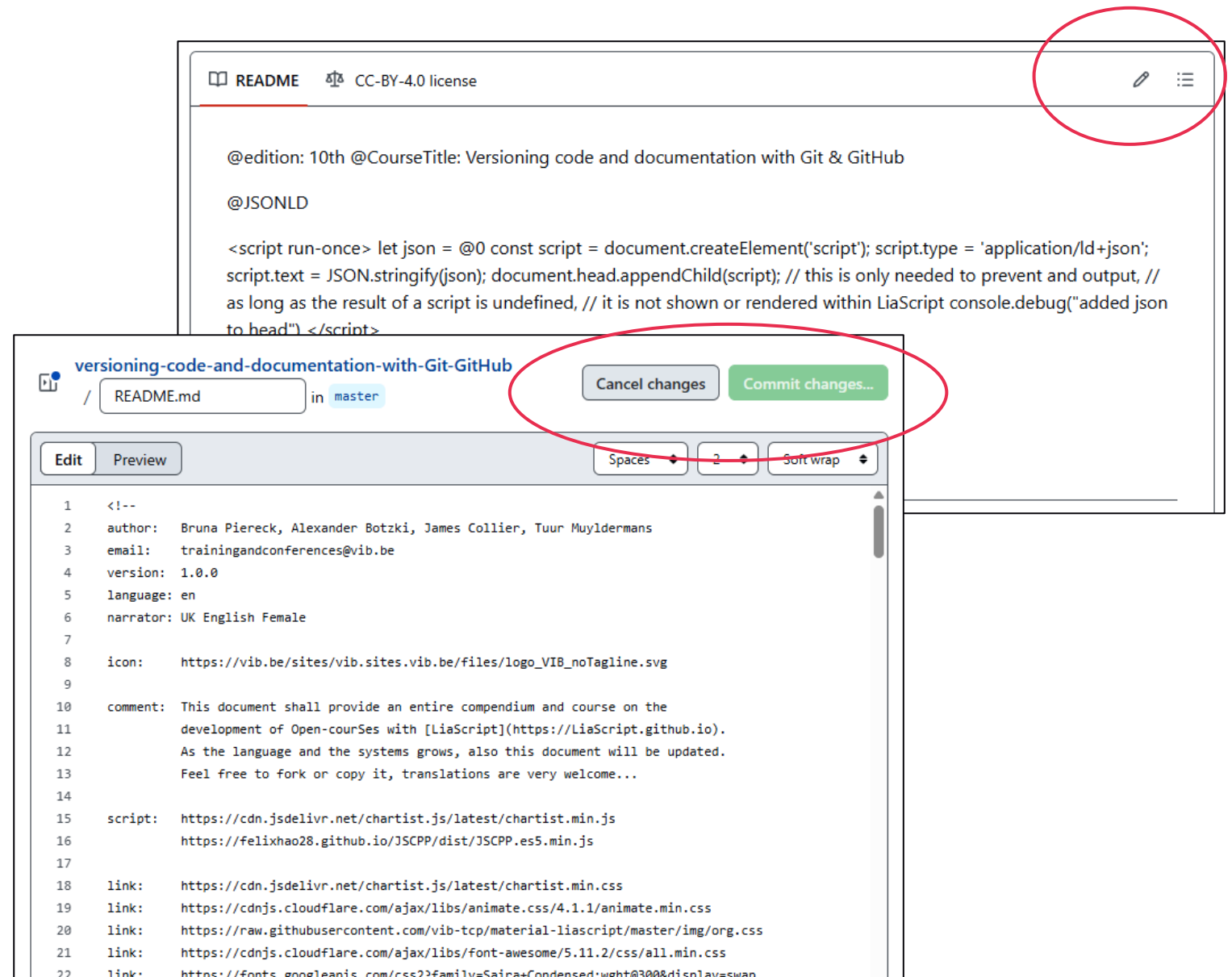
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



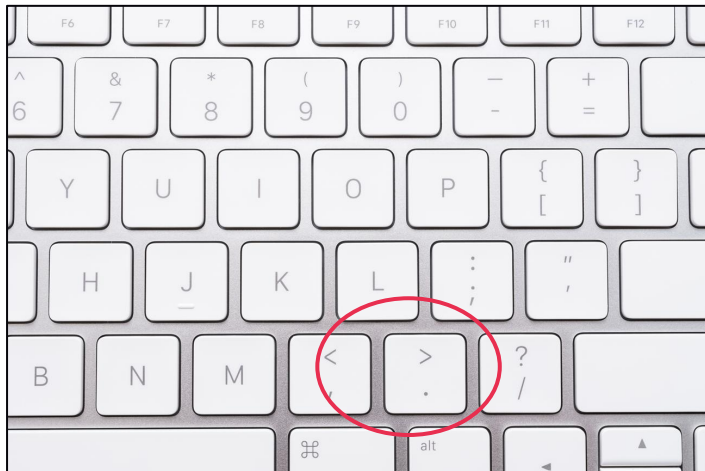
Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files

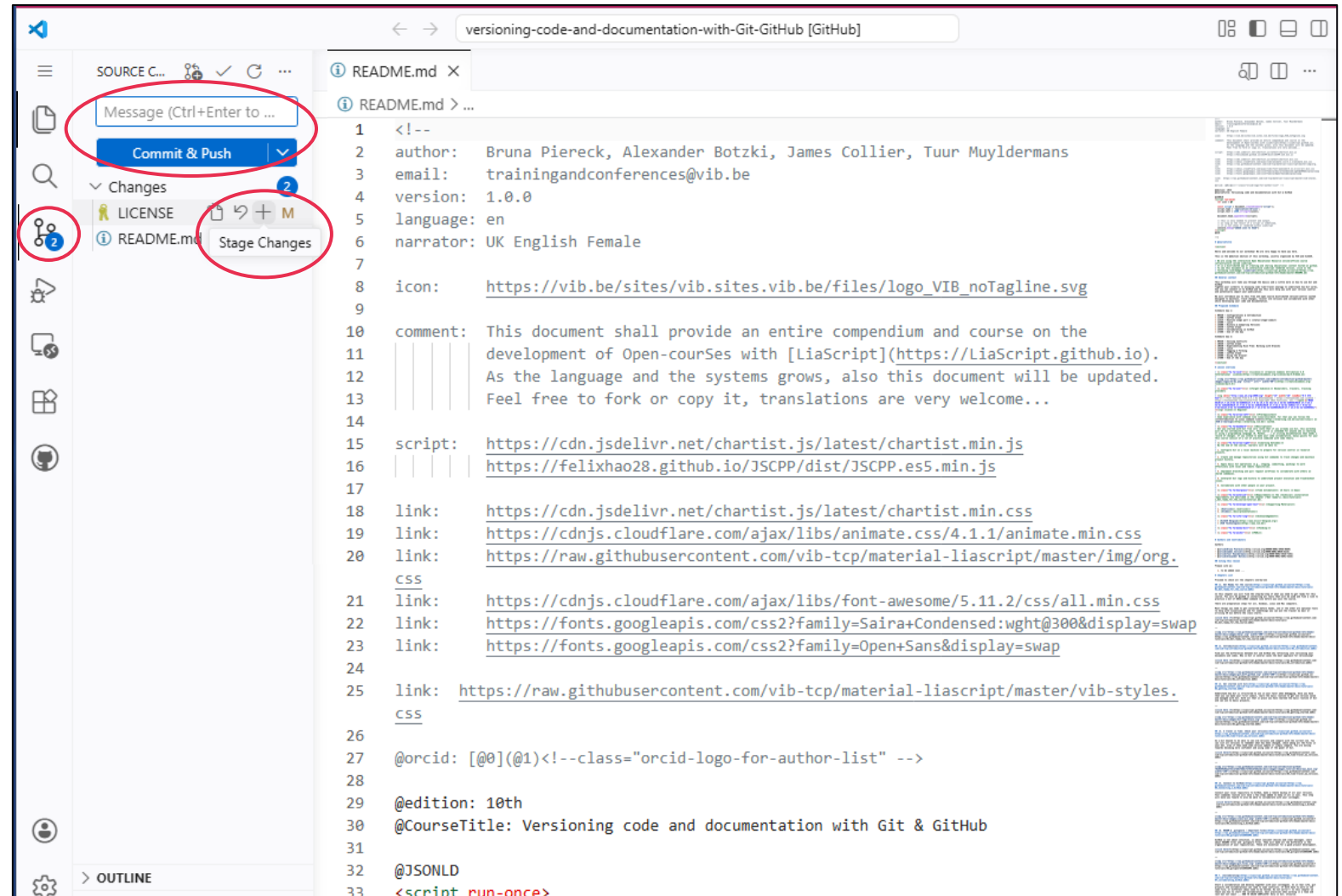


The image displays three screenshots of the GitHub interface, illustrating basic navigation and editing actions:

- Top Screenshot:** Shows the repository overview for "versioning-code-and-documentation-with-Git-GitHub". The "Code" button is circled in red, indicating the option to clone or download the repository.
- Middle Screenshot:** Shows the file view for "README.md". The "Edit file..." button in the toolbar is circled in red, indicating the option to edit the file.
- Bottom Screenshot:** Shows a modal menu for editing the file. The "github.dev" option is circled in red, indicating the choice to edit the file in the browser.

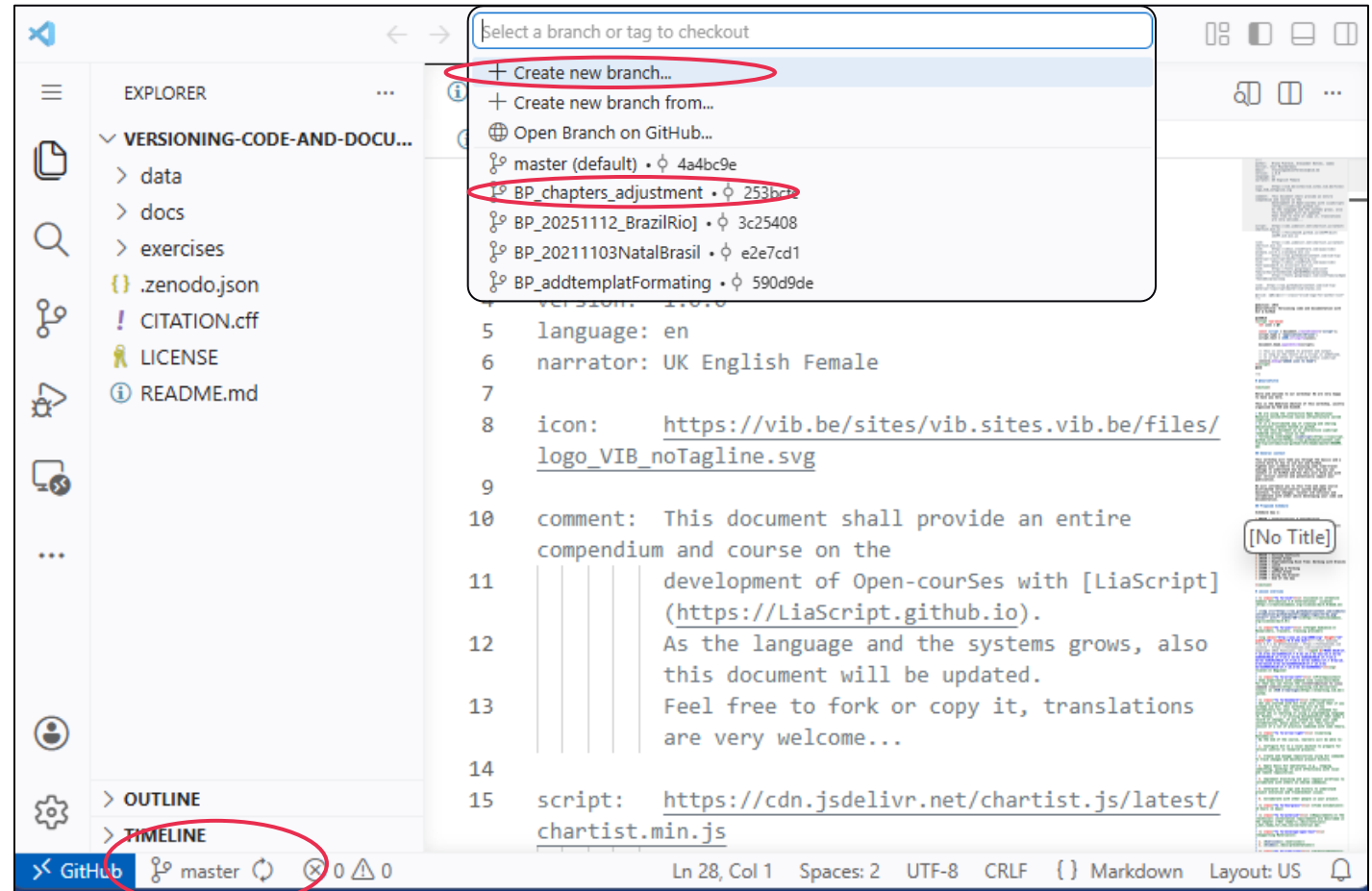
Basics on GitHub

- Check all branches
- Check all commits
- **Editing**
 - One-by-one
 - **Several files**
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



Branches

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases

The screenshot shows the GitHub interface for creating a new issue. The 'Issues' tab is highlighted with a red circle. The 'Create new issue' dialog box is open, showing the following fields and options:

- Add a title ***: A text input field with the placeholder 'Title'.
- Add a description**: A text area with the placeholder 'Type your description here...'. It includes a rich text editor toolbar with options for bold, italic, link, and other formatting.
- Assignees**: A dropdown menu showing 'No one - [Assign yourself](#)'.
- Labels**: A dropdown menu showing 'No labels'.
- Type**: A dropdown menu showing 'No type'.
- Projects**: A dropdown menu showing 'No projects'.
- Milestone**: A dropdown menu showing 'No milestone'.

At the bottom of the dialog, there are buttons for 'Create more', 'Cancel', and 'Create'.

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases

The screenshot shows a GitHub file view for a repository named "versioning-code-and-documentation-with-Git-GitHub". The file being viewed is "Git_QUIZ_intro2ndDay_En.md". The user "bpiereck" made a commit "Rename and reorganize folders to fit the new git-repo templ..." 4 months ago. The interface has tabs for "Preview", "Code", and "Blame", with "Code" selected. A toolbar at the top right includes icons for cloning, comparing, raw view, download, edit, and code blocks. The code content includes a title, mainfont, and a quiz question. Line 10 is highlighted in yellow, and a context menu is open over it. The menu options are: "Copy line", "Copy permalink", "View git blame", "Reference in new issue" (highlighted), "View file in GitHub.dev", and "View file in different branch/tag".

versioning-code-and-documentation-with-Git-GitHub / exercises
/ Git_QUIZ_intro2ndDay_En.md

bpiereck Rename and reorganize folders to fit the new git-repo templ... bf42468 · 4 months ago History

Preview Code Blame

1 ---
2 title: "Git & GitHub intro QUIZ for 2nd day English version"
3 mainfont: "Liberation Mono"
4 ---
5
6 # 1. What is True about Git / GitHub?
7
8 a. GitHub is software for versioning
9
10 b. Git is a website to keep backups of git projects
11
12 backups and collaborations
13
14
15
16
17
18
19
20
21
22
23 c. `git commit -m 'project description'`

Copy line
Copy permalink
View git blame
Reference in new issue
View file in GitHub.dev
View file in different branch/tag

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases

Creating ISSUES:

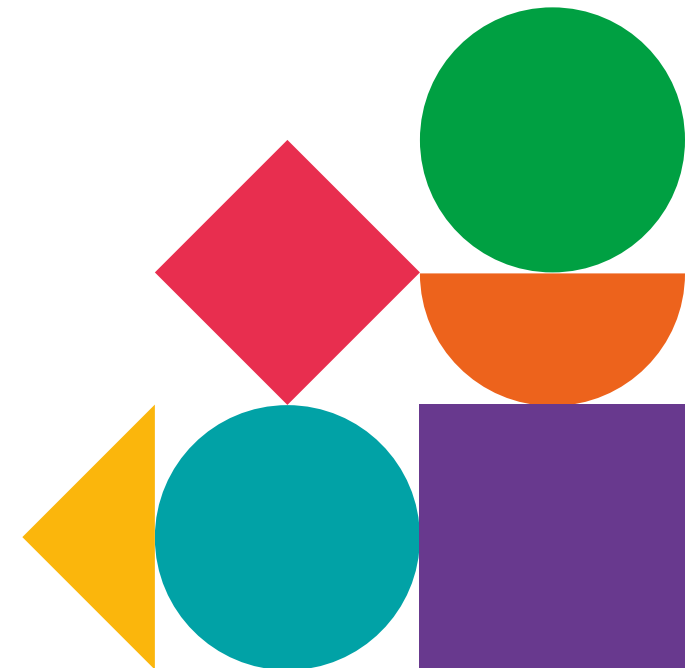
1. Descriptive title.
2. Explains the purpose of the issue
3. Include details that could help resolve the issue.

You can:

1. Create list of tasks
2. Create labels
3. Assign Issues to a collaborator
4. Exchange messages in a dedicated "chat"
5. Reference a line or paragraph

Basics on GitHub

- Check all branches
- Check all commits
- Editing
 - One-by-one
 - Several files
- Creating Issues
 - List of TODOs
 - List of bugs
- Tags and Releases



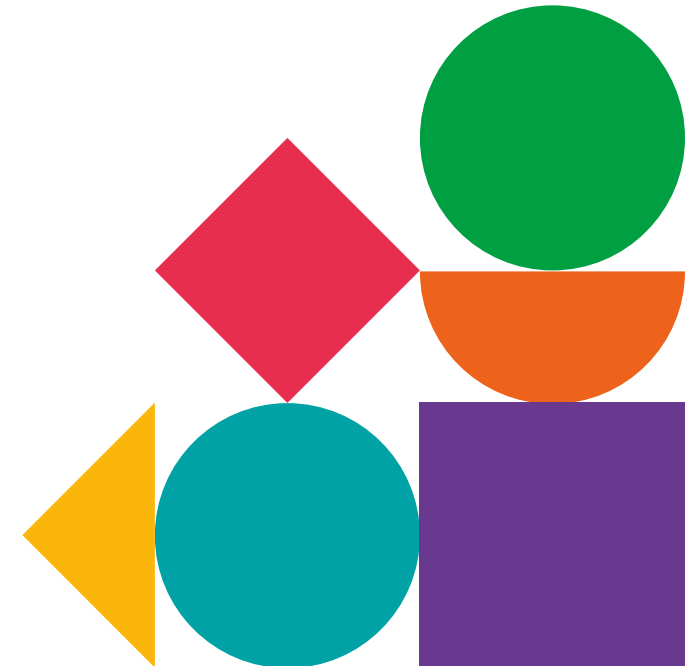
Tagging

Highlight
good versions

Tagging the commit

```
user$ git tag <name>
```

- Tag an important commit
 1. A good version
 2. A version for release
 3. A specific feature
 4. Experiment risk free
 5. Etc ..



Tagging the commit

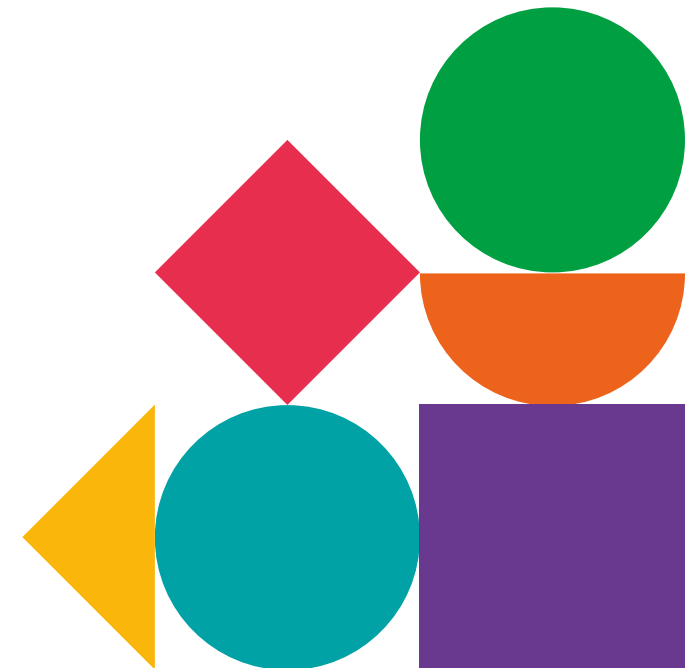
```
user$ git tag <name>
```

- Tag an important commit

1. A good version
2. A version for release
3. A specific feature
4. Experiment risk free
5. Etc ..

- Listing existing tags

```
user$ git tag  
user$ git tag -l  
user$ git tag --list
```



Tagging the commit

```
user$ git tag <name>
```

- Tag an important commit

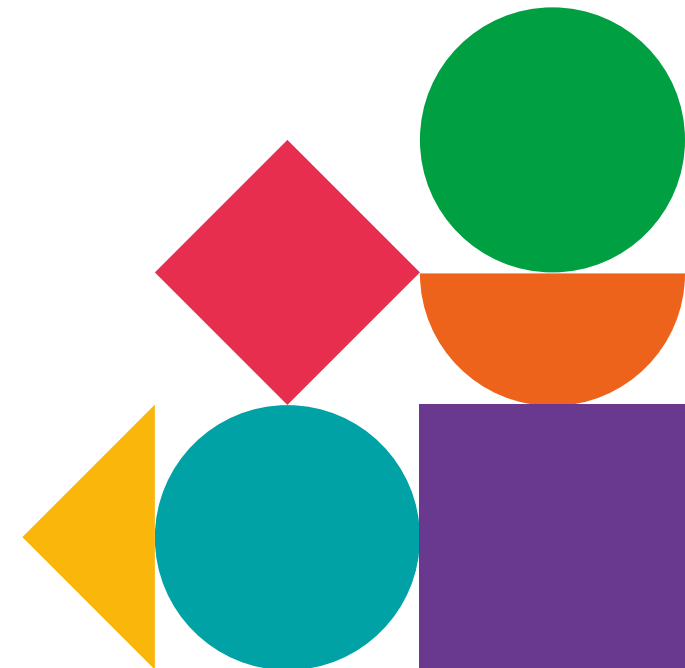
1. A good version
2. A version for release
3. A specific feature
4. Experiment risk free
5. Etc ..

- Listing existing tags

```
user$ git tag  
user$ git tag -l  
user$ git tag --list
```

- Tag an old commit

```
user$ git tag <name> <commit_ID>
```



ACTIVITY

9. Annotating

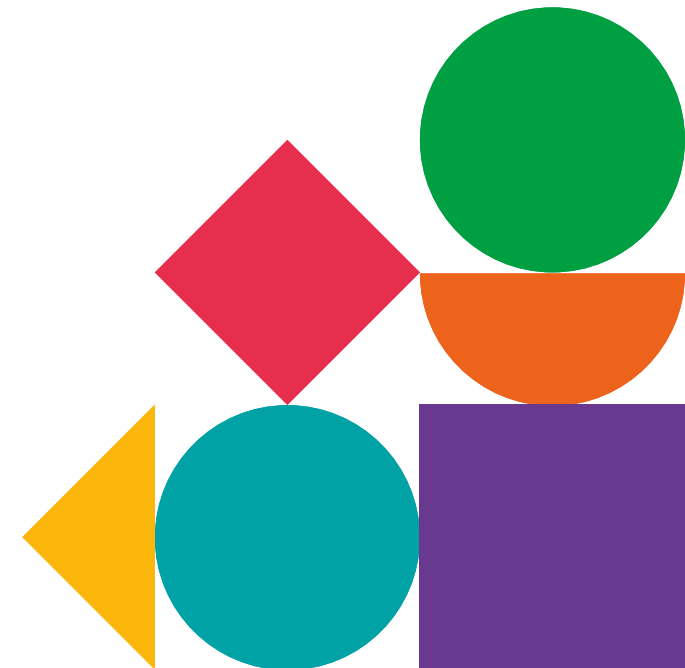
- Annotate in your documents about what is the tag used for
- What are the commands
- Commit

10. Creating tags

- Tag the last commit as “info_tag”
- Choose an old commit and create a tag for it
- List your tags

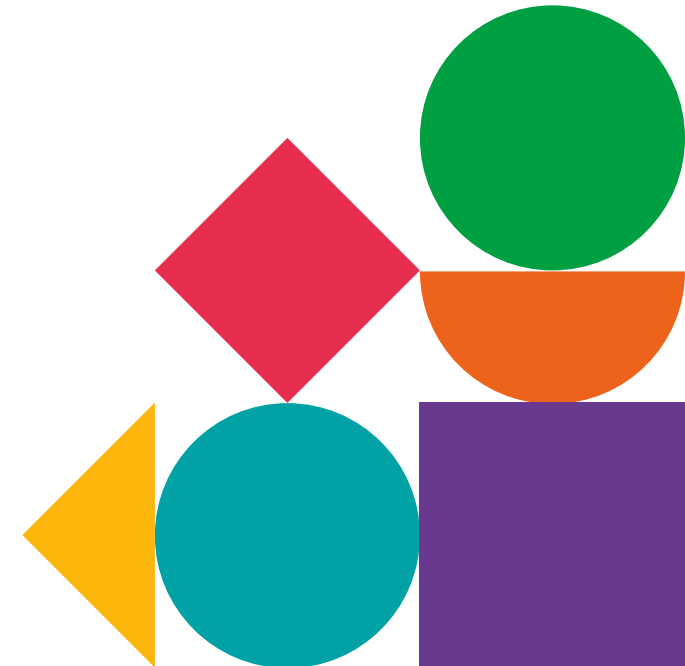
11. Challenge: Push your tags to GitHub

```
user$ git push --tags
```



ACTIVITY

12. Create a Release o GitHub

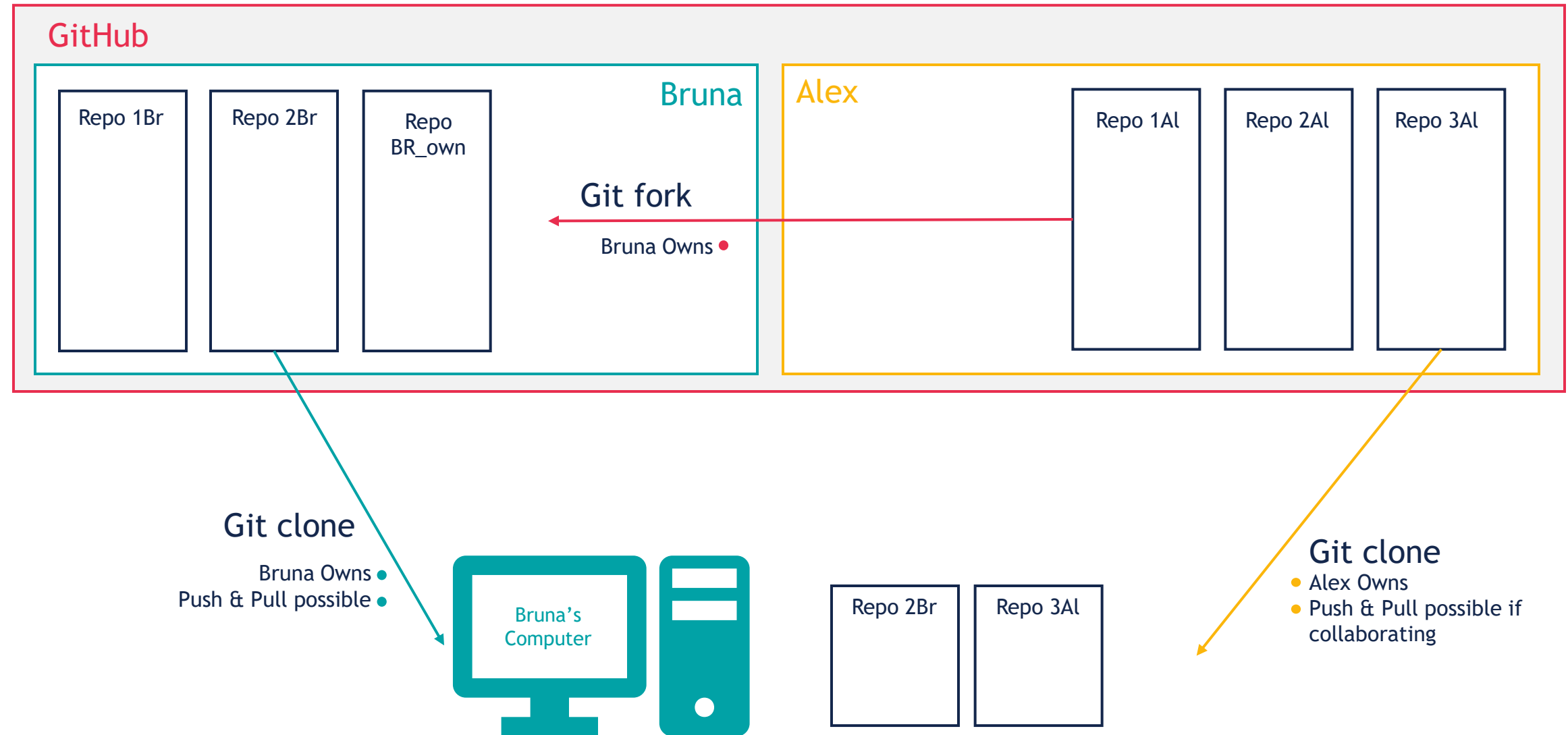


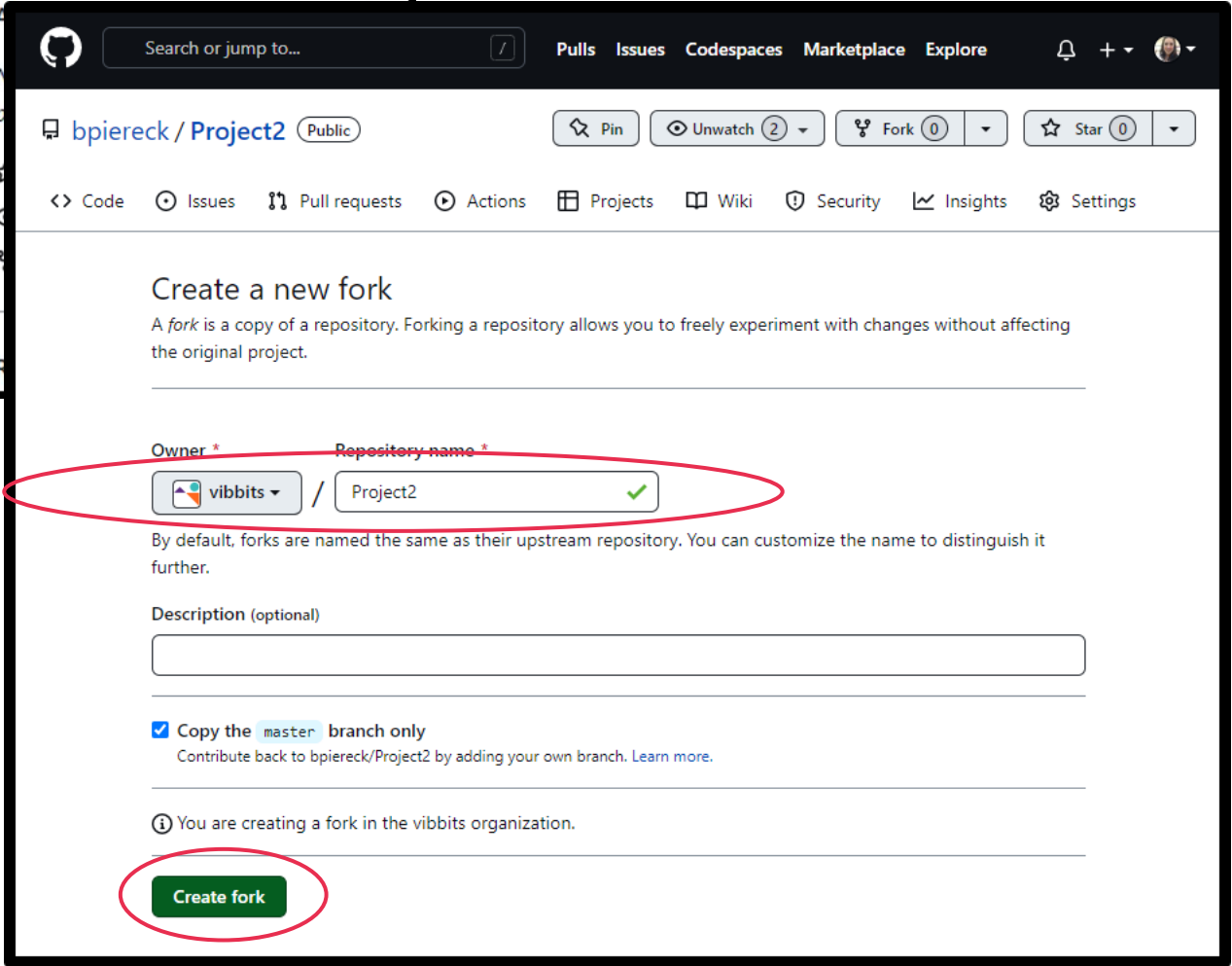
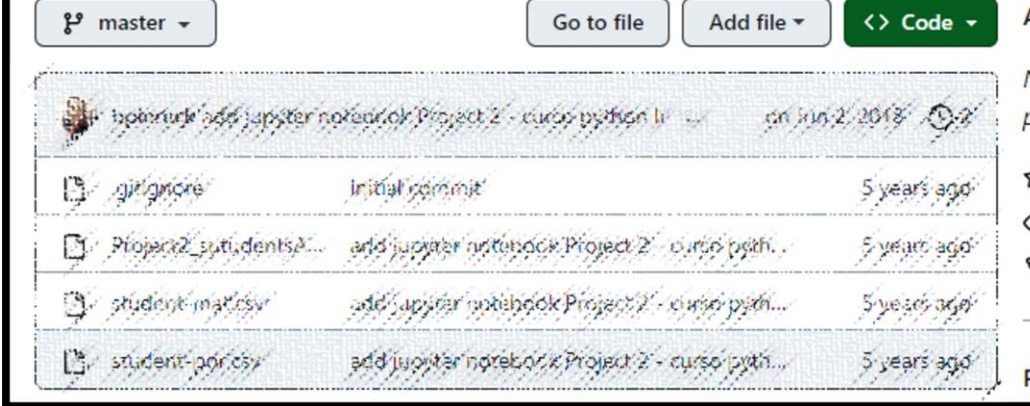
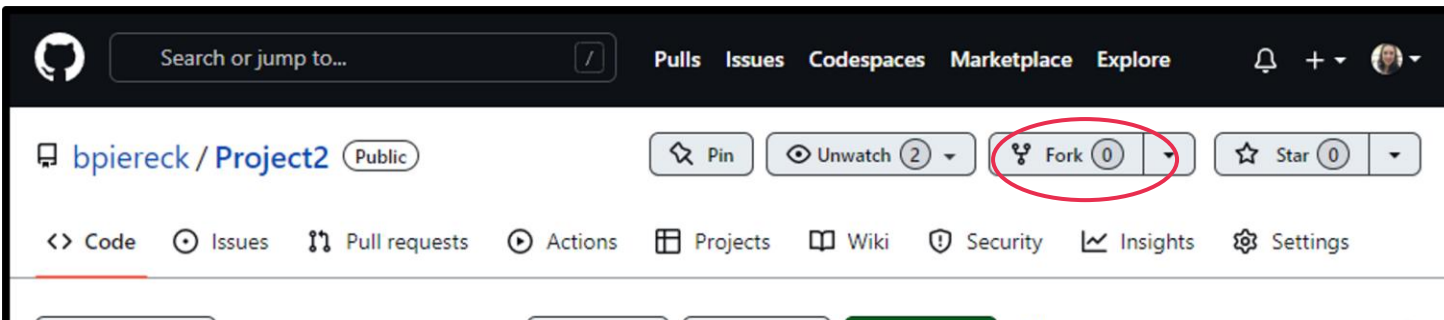
Forking



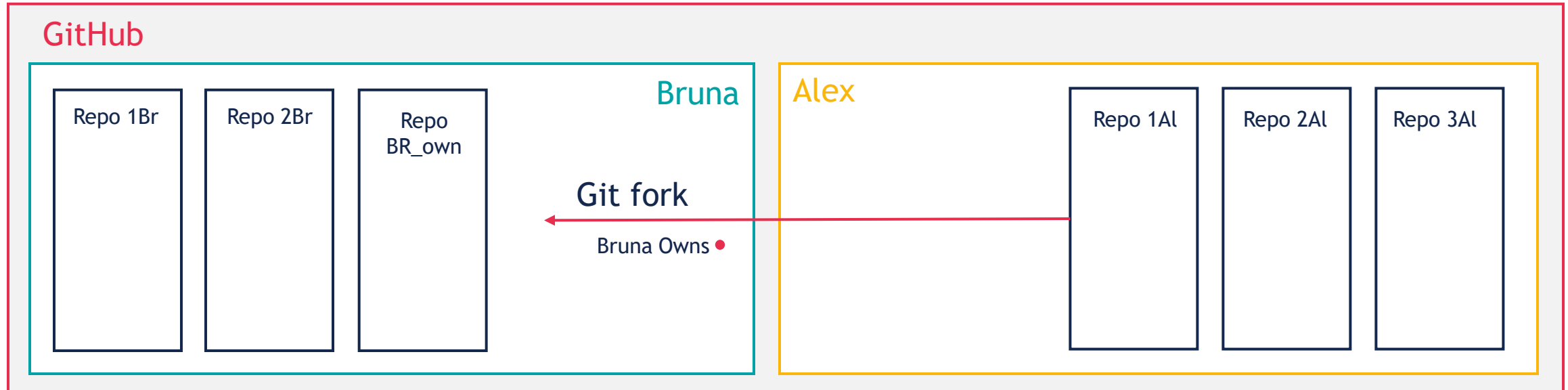
Copping
repositories

Fork vs Clone





Fork vs Clone



- Still connected
- Original creator can see it
- Changes can be incorporated
 - Both directions
 - Pull request

ACTIVITY

13. Fork race

- Wait for instructions to be a fair race!
- Add your name to the last line of the README file
- Commit (and wait)
- When instructed ask for a pull requested
 - A pull request must be evaluated and accepted

